

claude-windows / 05c32f87-3bfb-4a7f-861c-0050ac4169ac

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\05c32f87-3bfb-4a7f-861c-0050ac4169ac\subagents\workflows\wf_a8f43370-4cb\agent-a483bdc3be00cae5b.jsonl`  
- SHA-256: `831123cd2b913e8175bef50b060bc7fd3a1e05f47ce036786668febccb79aefb`  
- Source modified: `2026-07-07T15:17:30+00:00`  
- Imported at: `2026-07-09T08:32:23+00:00`  
- project: `wf_a8f43370-4cb`  
- session_id: `05c32f87-3bfb-4a7f-861c-0050ac4169ac`
```

Transcript

1. user (2026-07-07T15:07:26.827Z)

You are reviewing a two-repo bug fix BEFORE it merges into a HOSTED PRODUCTION Cloud Run control-plane.

THE BUG: On the hosted control-plane, POST /api/process with compute="local" ran the in-process segmenter ON THE CONTROL-PLANE, which bakes NO detector weights (only the worker Cloud Run Job GCSFuse-mounts weights at /gcs/models/...). It died deep with a cryptic "WASB weights not found at /opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar".
[COMPUTE] requested=local actual=in_process dispatched=None status=failed.

THE FIX (2 parts):

```
- BACKEND (repo undefined, branch undefined, PR #undefined):  
  backend/orchestration.py adds _WEIGHTS_BACKED_SEGMENTERS = {wasb_hybrid, tracknet_hybrid,  
    fusion_hybrid} and _local_detector_weights_present(cfg, seg_name). In _maybe_dispatch_remote  
  the  
    choice=="local" branch now fails fast via _failed(msg, False) when the segmenter is weights-  
  backed  
    AND _cloud_available(cfg) AND local weights are not readable; otherwise returns None (in-  
  process as  
    before). Plus 4 tests in tests/test_api_cloud_dispatch.py.  
- SPA (repo undefined, branch undefined, PR #undefined):  
  web/src/components/IngestPanel.tsx compute picker defaults to "cloud" when  
  capabilities.deployment.compute_target=="cloud_run" (else keeps the free "local" default).  
Plus 1 test.
```

AUTHORITATIVE CODE ? the working trees may NOT contain the change; read these snapshot files:

```
- undefined/backend.diff (backend commit diff)  
- undefined/orchestration_new.py (FULL post-change backend/orchestration.py)  
- undefined/test_api_cloud_dispatch_new.py (FULL post-change tests)  
- undefined/spa.diff (SPA commit diff)  
- undefined/IngestPanel_new.tsx (FULL post-change IngestPanel.tsx)
```

You MAY read other repo files for ground truth, e.g. under undefined:

```
  backend/pipeline/detectors/native_wasb_runner.py, wasb_runner.py, tracknet_runner.py;  
  backend/pipeline/segmenters/{wasb_hybrid,tracknet_hybrid,fusion_hybrid,trajectory_hybrid}.py;  
  backend/config/models.py; backend/main.py (the /api/capabilities handler ~line 364-410);  
and under undefined: web/src/api/types.ts, web/src/api/client.ts,  
web/src/components/IngestPanel.test.tsx.
```

You may also run: `gh pr diff undefined -R Khelsutra/badminton-highlight-indexer` and `gh pr diff undefined -R avidullu/khelsutra`.

Report ONLY real, concrete defects that would cause wrong behavior, a bad/incorrect merge, a regression, or a materially worse UX ? each with a SPECIFIC failure scenario (concrete config/inputs

? the wrong result). Verify claims against the actual code before reporting; do not invent style nits.

If nothing real exists in your dimension, return an empty findings array.

=== YOUR DIMENSION: cross-repo-contract ===

The SPA defaults to cloud on compute_target=="cloud_run"; the backend guardrail gates on _cloud_available(cfg) (which is TRUE when compute_target==cloud_run OR when CLOUD_RUN_JOB+GOOGLE_CLOUD_PROJECT+storage.output_root are all set). Enumerate the deployment matrix and find any

dead-end, contradiction, or confusing UX:

(a) hosted control-plane: compute_target=cloud_run, cloud_available=true, NO local weights;

(b) local appliance: compute_target=local_gpu/"", cloud_available=false, HAS local weights;

(c) local-default server ENV-WIRED for cloud: compute_target="", but

CLOUD_RUN_JOB+project+bucket set

? cloud_available=true; may or may not have local weights.

For each: what does the SPA picker default to and SEND, and what does the backend do? Is the backend

message "Choose 'Run in cloud'" always actionable (does the SPA actually offer a working cloud path in

that state)? In case (c) the SPA keeps LOCAL default (compute_target?cloud_run) and sends compute=local ? if that box lacks weights, the backend now fails fast: is that the right outcome, and

is the message coherent? Any case where the user is stuck with no working option, or where the SPA

sends cloud but the backend can't dispatch, or vice-versa?

2. assistant (2026-07-07T15:07:33.509Z)

I'll start by exploring the environment to find the snapshot files and the actual repo layout.

3. user (2026-07-07T15:07:36.412Z)

total 62

```
drwxr-xr-x 1 avidu 197609  0 Jul  5 12:57 .
drwxr-xr-x 1 avidu 197609  0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609  0 Jun 22 11:12 .claude
drwxr-xr-x 1 avidu 197609  0 Jul  7 20:33 .git
-rw-r--r-- 1 avidu 197609 297 Jun 20 23:27 .gitattributes
drwxr-xr-x 1 avidu 197609  0 Jun 20 12:43 .github
-rw-r--r-- 1 avidu 197609 323 Jun 22 14:57 .gitignore
drwxr-xr-x 1 avidu 197609  0 Jul  2 21:13 deploy
drwxr-xr-x 1 avidu 197609  0 Jul  2 21:13 docs
drwxr-xr-x 1 avidu 197609  0 Jun 21 23:55 locales
-rw-r--r-- 1 avidu 197609 9773 Jul  2 21:13 NEXT_STEPS.md
drwxr-xr-x 1 avidu 197609  0 Jun 21 15:38 node_modules
-rw-r--r-- 1 avidu 197609 1469 Jun 23 00:19 README.md
drwxr-xr-x 1 avidu 197609  0 Jun 23 00:19 site
drwxr-xr-x 1 avidu 197609  0 Jul  7 19:45 web
```

---PARENT---

total 108

```
drwxr-xr-x 1 avidu 197609  0 Jul  7 19:57 .
drwxr-xr-x 1 avidu 197609  0 Jul  7 20:24 ..
drwxr-xr-x 1 avidu 197609  0 Jun 23 09:53 Annotation Setup
-rw-r--r-- 1 avidu 197609 905 Jul  2 21:20 AUDIT_REPORT_khelsutra-guru_2026-07-02.md
drwxr-xr-x 1 avidu 197609  0 Jul  7 20:14 badminton-highlight-indexer
-rw-r--r-- 1 avidu 197609 1222 Jul  2 21:35 bash.exe.stackdump
drwxr-xr-x 1 avidu 197609  0 Jul  7 20:34 bhi-wt-worker-reval
drwxr-xr-x 1 avidu 197609  0 Jul  5 12:57 khelsutra
drwxr-xr-x 1 avidu 197609  0 Jul  5 18:47 khelsutra-screencast
drwxr-xr-x 1 avidu 197609  0 Jul  5 13:44 marketing
drwxr-xr-x 1 avidu 197609  0 Jul  5 13:43 pr-158-screenshots
drwxr-xr-x 1 avidu 197609  0 Jun 22 15:03 rally-annotator
drwxr-xr-x 1 avidu 197609  0 Jul  4 06:58 sports-data-collector
drwxr-xr-x 1 avidu 197609  0 Jul  2 19:46 sports-obsreport
```

---PROJECTS---

total 10136

```
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:24 .
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:36 ..
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:12 .tmp.drivedownload
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:37 .tmp.driveupload
drwxr-xr-x 1 avidu 197609 0 Jun 22 09:41 _p3_scratch
drwxr-xr-x 1 avidu 197609 0 Jul 7 09:28 _wt-pr58
drwxr-xr-x 1 avidu 197609 0 Jul 6 10:22 Agent Sessions
drwxr-xr-x 1 avidu 197609 0 Jul 7 19:25 Agent Sessions - baseline-lint
drwxr-xr-x 1 avidu 197609 0 Jul 7 18:25 Agent Sessions - baseline-schema-doc
drwxr-xr-x 1 avidu 197609 0 Jul 7 17:05 Agent Sessions - first-user-setup-ux
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:37 Agent Sessions - handoff-proposals
drwxr-xr-x 1 avidu 197609 0 Jul 7 18:42 Agent Sessions - handoffs-audit
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:11 Agent Sessions - handoffs-index-feed
drwxr-xr-x 1 avidu 197609 0 Jul 7 18:09 Agent Sessions - knowledge-replay-design-plan
drwxr-xr-x 1 avidu 197609 0 Jul 7 19:09 Agent Sessions - project-page-upserts
drwxr-xr-x 1 avidu 197609 0 Jul 7 19:50 Agent Sessions - trace-reference-validation
drwxr-xr-x 1 avidu 197609 0 Jul 2 11:41 badminton-highlight-indexer
drwxr-xr-x 1 avidu 197609 0 Jul 7 19:57 khelsutra-guru
drwxr-xr-x 1 avidu 197609 0 Jun 29 14:02 muneem-ai
drwxr-xr-x 1 avidu 197609 0 Jul 7 10:23 Telegram
drwxr-xr-x 1 avidu 197609 0 Jun 16 09:55 Vairagi
```

4. user (2026-07-07T15:07:53.541Z)

C:/Users/avidu/AppData/Local/Temp/backend.diff

C:/Users/avidu/AppData/Local/Temp/spa.diff

5. assistant (2026-07-07T15:07:58.882Z)

The snapshot files are in `C:/Users/avidu/AppData/Local/Temp/`. Let me find all of them and read the key ones.

6. user (2026-07-07T15:08:04.083Z)

```
-rw-r--r-- 1 avidu 197609 11027 Jul 7 20:34 C:/Users/avidu/AppData/Local/Temp/backend.diff
-rw-r--r-- 1 avidu 197609 53127 Jul 4 06:14 C:/Users/avidu/AppData/Local/Temp/pr458.diff
-rw-r--r-- 1 avidu 197609 53127 Jul 4 06:15 C:/Users/avidu/AppData/Local/Temp/pr458_full.diff
-rw-r--r-- 1 avidu 197609 5007 Jul 7 20:34 C:/Users/avidu/AppData/Local/Temp/spa.diff
===ALL TEMP TOP-LEVEL (non-dir) files matching===
C:/Users/avidu/AppData/Local/Temp/backend.diff
C:/Users/avidu/AppData/Local/Temp/pr458.diff
C:/Users/avidu/AppData/Local/Temp/pr458_full.diff
C:/Users/avidu/AppData/Local/Temp/spa.diff
```

7. user (2026-07-07T15:08:14.737Z)

```
1      commit 4565c7436ccf0b4e30f68f01efe6082eff371a5d
2      Author: Avi Dullu <avi.dullu@gmail.com>
3      Date:   Tue Jul 7 19:56:22 2026 +0530
4
5          fix(serving): fail fast when compute=local forces weightless in-process detection
6
7          A hosted cloud-serving control-plane bakes NO detector weights into its image
8          (only the worker Cloud Run Job GCSFuse-mounts them at /gcs/models/...). So a
9          POST /api/process with compute="local" for a weights-backed trajectory detector
10         (wasb_hybrid / tracknet_hybrid / fusion_hybrid) returned None from
11         _maybe_dispatch_remote and ran the in-process segmenter ON THE CONTROL-PLANE,
12         which then died deep in the healthcheck with the confusing
13         "WASB weights not found at /opt/models/WASB-SBDT/..." (observed live 2026-07-07,
14         job f29d41d0?: [COMPUTE] requested=local actual=in_process ... status=failed).
15
16         Guardrail: on a cloud-serving-capable server (_cloud_available) whose local
```

```

17     weights for the requested detector aren't readable, a forced compute="local"
18     now fails FAST with an actionable message ("this hosted server can't run
19     detection locally ... Choose 'Run in cloud'.") ? mirroring the existing
20     compute="cloud"-not-available reject ? instead of the silent in-process trap.
21     A truly-local box, or a cloud-wired box that actually has the weights on disk,
22     is unaffected (the weights-present check is the escape hatch); a weight-free
23     segmenter (motion) is out of scope. Honest provenance: path=not_run,
24     requested=local, dispatched=False.
25
26     Covered by tests/test_api_cloud_dispatch.py (fail-fast, weights-present escape
27     hatch, motion unaffected, pure-local not guardrailed).
28
29     Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
30
31     diff --git a/backend/orchestration.py b/backend/orchestration.py
32     index 5e9f510..bfe39f5 100644
33     --- a/backend/orchestration.py
34     +++ b/backend/orchestration.py
35     @@ -155,6 +155,38 @@ def _ingest_remote_segments(
36         return n
37
38
39     ## Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS
40     box. A hosted
41     ## cloud-serving control-plane bakes NO detector weights into its image (only the worker
42     Cloud Run Job
43     ## GCSFuse-mounts them), so forcing compute="local" for one of these there can never
44     succeed ? the
45     ## guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
46     deep
47     ## "WASB weights not found". (Names are the registered segmenter identities; see
48     backend/pipeline/segmenters.)
49     +_WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
50     "fusion_hybrid"})
51     +
52     +
53     +def _local_detector_weights_present(cfg, seg_name: str) -> bool:
54     +    """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
55     present + readable
56     +    on THIS box. Mirrors the native/WSL runners' resolution (env override ?
57     ``indexing.<det>.weights_path``)
58     +    and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
59     that would
60     +    otherwise fail deep in the run. Fusion follows its configured substrate (WASB
61     default / TrackNet).
62     +    A non-weights segmenter returns True (nothing local to check)."""
63     +    name = (seg_name or "").strip().lower()
64     +    if name not in _WEIGHTS_BACKED_SEGMENTERS:
65     +        return True
66     +    idx = getattr(cfg, "indexing", None)
67     +    substrate = (
68     +        getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
69     +    ).strip().lower()
70     +    if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
71     "tracknet"):
72     +        path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
73     +    else: # wasb_hybrid, or fusion on the (default) WASB substrate
74     +        path = (
75     +            os.environ.get("WASB_WEIGHTS")
76     +            or getattr(getattr(idx, "wasb", None), "weights_path", "")
77     +            or ""
78     +        )
79     +    return bool(path) and os.path.exists(os.path.expanduser(path))

```

```

71     def _cloud_available(cfg) -> bool:
72         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
73 ).
74     @@ -446,6 +478,23 @@ def _maybe_dispatch_remote(
75         )
76         choice = ""
77         if choice == "local":
78             + # A hosted cloud-serving control-plane bakes NO detector weights into its image
79             + # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
80             + # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
81             + # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail
82             + # actionable message instead of that silent trap. A truly-local box (not cloud-
83             + # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it
84             + # below exactly as before (the weights-present check is the escape hatch).
85             + if (
86                 + (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
87                 + and _cloud_available(cfg)
88                 + and not _local_detector_weights_present(cfg, seg_name)
89             ):
90                 return _failed(
91                     + f"this hosted server can't run detection locally ? the '{seg_name}'
92                     + "live only on the cloud worker, not on the control-plane. Choose 'Run
93                     + False,
94                     + )
95                 return None # explicit "run on my machine" ? in-process regardless of the
96                 if choice == "cloud":
97                     if not _cloud_available(cfg):
98                         diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
99                         index 8382234..7993b7f 100644
100                         --- a/tests/test_api_cloud_dispatch.py
101                         +++ b/tests/test_api_cloud_dispatch.py
102                 @@ -721,6 +721,114 @@ def
103                 test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypa
104                     assert all(_meta(s)["source"] == "cloud_run" for s in segs)
105
106             + # --- hosted control-plane guardrail: compute='local' for a weights-backed detector
107             + # A hosted cloud-serving control-plane bakes NO detector weights (only the worker Cloud
108             + # them), so a forced compute='local' for wasb/tracknet/fusion can't run in-process
109             + # FAST with an actionable "run in the cloud" message ? not fall through to the in-
110             + # that dies deep with a cryptic "WASB weights not found" (observed live: job f29d41d0?,
111             + # 2026-07-07).
112             +
113             + def test_local_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
114                 + """compute='local' + a weights-backed detector on a cloud_run server with NO local
115                 + a clear pre-dispatch failure (never dispatched, never the cryptic deep 'weights not
116                 + _db, _bucket, _mp = cloud_env
117                 + monkeypatch.delenv("WASB_WEIGHTS", raising=False)

```

```

118 +     m.global_config.indexing.wasb.weights_path =
119 +         "/nonexistent/wasb_badminton_best.pth.tar"
120 +     fake = _FakeCloudClient()
121 +     _inject_fake(monkeypatch, fake)
122 +
123 +     out = m._execute_processing(
124 +         m.ProcessRequest(
125 +             video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
126 +         )
127 +     )
128 +     assert out["status"] == "failed"
129 +     assert "can't run detection locally" in out["message"]
130 +     assert "Run in cloud" in out["message"]
131 +     assert "weights not found" not in out["message"] # NOT the cryptic deep failure
132 +     assert fake.calls == [] # never dispatched
133 +     # Honest provenance: requested local, ran NOWHERE (pre-dispatch reject).
134 +     assert out["compute"]["requested"] == "local"
135 +     assert out["compute"]["dispatched"] is False
136 +     assert out["compute"]["path"] == "not_run"
137 +
138 + def test_local_on_cloud_server_with_weights_present_runs_in_process(
139 +     cloud_env, tmp_path, monkeypatch
140 + ):
141 +     """Escape hatch: the guardrail must NOT over-block ? on a cloud-wired box that
142 +     ACTUALLY has the
143 +     detector weights on disk, compute='local' still runs the in-process segmenter (no
144 +     dispatch)."""
145 +     _db, _bucket, _mp = cloud_env
146 +     weights = tmp_path / "wasb_badminton_best.pth.tar"
147 +     weights.write_bytes(b"WEIGHTS")
148 +     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
149 +     m.global_config.indexing.wasb.weights_path = str(weights)
150 +     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
151 +     fake = _FakeCloudClient()
152 +     _inject_fake(monkeypatch, fake)
153 +
154 +     out = m._execute_processing(
155 +         m.ProcessRequest(
156 +             video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
157 +         )
158 +     )
159 +     assert out["status"] == "success"
160 +     assert fake.calls == [] # in-process, not dispatched
161 +     assert out["compute"]["path"] == "in_process"
162 +
163 + def test_local_non_weights_segmenter_on_cloud_server_still_runs_in_process(
164 +     cloud_env, monkeypatch
165 + ):
166 +     """The guardrail is scoped to weights-backed detectors: a weight-free segmenter
167 +     (motion) with
168 +     compute='local' on a cloud server is unaffected ? it runs in-process, never fails
169 +     fast."""
170 +     _db, _bucket, _mp = cloud_env
171 +     m.global_config.indexing.wasb.weights_path =
172 +         "/nonexistent/should_not_matter.pth.tar"
173 +     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
174 +     fake = _FakeCloudClient()
175 +     _inject_fake(monkeypatch, fake)
176 +
177 +     out = m._execute_processing(
178 +         m.ProcessRequest(
179 +             video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
180 +         )
181 +     )

```

```

177 + )
178 + assert out["status"] == "success"
179 + assert fake.calls == []
180 + assert out["compute"]["path"] == "in_process"
181 +
182 +
183 +def test_local_weights_backed_on_pure_local_server_is_not_guardrailed(
184 +    tmp_path, monkeypatch
185 +):
186 +    """The guardrail only fires on a cloud-capable server. On a pure-local server (no
cloud), a
187 +    weights-backed compute='local' request falls through to the in-process segmenter as
before ? the
188 +    'run in the cloud' message would be nonsense there (no cloud to fall back to)."""
189 +    db = Database(str(tmp_path / "t.db")) # noqa: F841
190 +    monkeypatch.setattr(m, "db_instance", db)
191 +    cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}}) # noqa:
F841
192 +    monkeypatch.setattr(m, "global_config", cfg)
193 +    monkeypatch.setattr(
194 +        m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
195 +    )
196 +    monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
197 +    monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
198 +    monkeypatch.delenv("WASB_WEIGHTS", raising=False)
199 +    cfg.indexing.wasb.weights_path = "/nonexistent/wasb.pth.tar"
200 +    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
201 +    video = tmp_path / "v.mp4"
202 +    video.write_bytes(b"hashable-bytes")
203 +
204 +    out = m._execute_processing(
205 +        m.ProcessRequest(
206 +            video_path=str(video), segmenter="wasb_hybrid", compute="local"
207 +        )
208 +    )
209 +    # Not the hosted fast-fail ? it ran in-process (the local box owns its own weights
story).
210 +    assert out["status"] == "success"
211 +    assert out["compute"]["path"] == "in_process"
212 +
213 +
214 +    # --- compute-path PROVENANCE (the validation method: prove which backend ran)
-----
215
216
217

```

8. user (2026-07-07T15:08:17.081Z)

```

===SCRATCHPAD===
total 156
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:35 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:31 ..
-rw-r--r-- 1 avidu 197609 11027 Jul  7 20:35 backend.diff
-rw-r--r-- 1 avidu 197609 29098 Jul  7 20:35 IngestPanel_new.tsx
-rw-r--r-- 1 avidu 197609 55735 Jul  7 20:35 orchestration_new.py
-rw-r--r-- 1 avidu 197609  5007 Jul  7 20:35 spa.diff
-rw-r--r-- 1 avidu 197609 42525 Jul  7 20:35 test_api_cloud_dispatch_new.py
===any *_new anywhere in temp claude===
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/backend.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/IngestPanel_new.tsx
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/orchestration_new.py

```

C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/spa.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/test_api_cloud_dispatch_new.py
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr35.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr37.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr38.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr39.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr50.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr51.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr52.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr53.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr54.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr55.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr57.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr58.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/pr55.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_full.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/edc346c2-92fc-4ad1-b556-c7ab955a5613/scratchpad/pr158.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/edc346c2-92fc-4ad1-b556-c7ab955a5613/scratchpad/service_new.yaml

9. assistant (2026-07-07T15:08:24.215Z)

Now let me read the full post-change files and the SPA diff.

10. user (2026-07-07T15:08:25.246Z)

```
1      commit 1470e2acdea4f2b478476150da61e844dce0a8e9
2      Author: Avi Dullu <avi.dullu@gmail.com>
3      Date:   Tue Jul 7 19:57:35 2026 +0530
4
5          fix(ingest): default the compute picker to CLOUD on a hosted control-plane
6
7          On a hosted cloud-serving control-plane (capabilities.deployment.compute_target
8          == "cloud_run") the engine bakes NO detector weights, so in-process LOCAL
9          detection always fails ("WASB weights not found"). The picker still defaulted to
10         LOCAL and sent compute="local", which the engine ran on the weightless
11         control-plane -> a confusing failure.
12
13         Default the picker to CLOUD when the server advertises compute_target=cloud_run
14         (cloud_available too), so the SPA never sends compute=local by default on the
15         hosted build. The free-LOCAL default is kept for a local appliance that merely
16         also offers cloud (compute_target != cloud_run). The cloud cost-acknowledgement
17         gate is unchanged (no paid dispatch without consent).
18
19         Pairs with the engine-side fast-fail guardrail (badminton-highlight-indexer
20         _maybe_dispatch_remote) for anyone who still explicitly picks LOCAL on the
```



```

21         hosted build. Covered by an IngestPanel test.
22
23         Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
24
25         diff --git a/web/src/components/IngestPanel.test.tsx
b/web/src/components/IngestPanel.test.tsx
26         index 685aa1b..b39e4f1 100644
27         --- a/web/src/components/IngestPanel.test.tsx
28         +++ b/web/src/components/IngestPanel.test.tsx
29         @@ -185,6 +185,30 @@ describe("IngestPanel", () => {
30             expect(bodies[0].compute).toBe("local");
31         });
32
33         + it("defaults to CLOUD (never sends compute=local) on a hosted cloud_run control-
plane", async () => {
34             + // A hosted control-plane bakes no detector weights, so LOCAL can't run there ? the
picker must
35             + // default to CLOUD. The cost gate still applies (accept it, then the sole submit
is compute=cloud).
36             + const bodies: Record<string, unknown>[] = [];
37             + vi.stubGlobal(
38             +     "fetch",
39             +     fetchAll({
40             +         caps: { deployment: { cloud_available: true, compute_target: "cloud_run" } },
41             +         onProcessBody: (b) => bodies.push(b),
42             +         jobs: [{ job_id: "j", status: "done", video_id: "vidH", result: { status:
"success", video_id: "vidH" } }],
43             +     }),
44             + );
45             + render(<IngestPanel onLoaded={vi.fn()} />);
46             + await screen.findByTestId("compute-picker");
47             + // Defaulted to cloud ? the cost note + ack are shown up front (LOCAL would hide
them).
48             + expect(screen.getByTestId("compute-cost")).toBeTruthy();
49             + typeUri("gs://bucket/match.mp4");
50             + fireEvent.click(screen.getByRole("checkbox")); // accept the cost gate
51             + clickProcess();
52             + await act(async () => {});
53             + expect(bodies).toHaveLength(1);
54             + expect(bodies[0].compute).toBe("cloud"); // never "local" by default on the hosted
build
55             + });
56             +
57             it("shows a pasted host-path basename while processing and clears it on cancel",
async () => {
58                 vi.useFakeTimers();
59                 vi.stubGlobal(
60                 diff --git a/web/src/components/IngestPanel.tsx b/web/src/components/IngestPanel.tsx
61                 index 38ca3a9..edfd5d9 100644
62                 --- a/web/src/components/IngestPanel.tsx
63                 +++ b/web/src/components/IngestPanel.tsx
64                 @@ -81,7 +81,9 @@ export function IngestPanel({ onLoaded, onBusyChange }: Props) {
65
66                 // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to
a cloud GPU
67                 // (capabilities.deployment.cloud_available), let the user choose where DETECT runs.
Default LOCAL
68                 - // (free, no surprise cost); CLOUD requires an explicit cost acknowledgement before
it can run.
69                 + // (free, no surprise cost) EXCEPT on a hosted cloud-serving control-plane
(compute_target=cloud_run):
70                 + // there the control-plane bakes no detector weights, so LOCAL can never run ?
default CLOUD instead
71                 + // (set from capabilities below), which requires an explicit cost acknowledgement
before it can run.

```

```

72     const [cloudAvailable, setCloudAvailable] = useState(false);
73     const [compute, setCompute] = useState<Compute>("local");
74     const [cloudAck, setCloudAck] = useState(false);
75     @@ -101,6 +103,13 @@ export function IngestPanel({ onLoaded, onBusyChange }: Props) {
76         .then((c) => {
77             if (!alive) return;
78             setCloudAvailable(c.deployment?.cloud_available === true);
79         +        // Hosted cloud-serving control-plane (compute_target=cloud_run) bakes NO
detector weights, so
80         +        // in-process LOCAL detection there always fails ("weights not found") ?
default the picker to
81         +        // CLOUD so the SPA never sends compute=local by default on the hosted build. A
local appliance
82         +        // that merely also offers cloud (compute_target?cloud_run) keeps the free
LOCAL default.
83         +        if (c.deployment?.cloud_available === true && c.deployment?.compute_target ===
"cloud_run") {
84             +            setCompute("cloud");
85             +        }
86             setBackends(c.storage?.backends ?? []);
87             setMaxUploadBytes(
88                 typeof c.max_upload_mb === "number" && c.max_upload_mb > 0 ? c.max_upload_mb
* 1024 * 1024 : undefined,
89

```

11. user (2026-07-07T15:08:27.564Z)

```

1         // The front-half ingest screen (appliance S1/S2; tracker B-P2 + T4). Two ways to point
the tool at a
2         // video: UPLOAD a local file (primary ? alpha users have a file, not a URI; chunked to
the engine's
3         // /api/upload/* endpoints) or paste a `gs://` bucket URI / host path (secondary). Both
converge on
4         // the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id
to App. Honest
5         // by construction: the engine reports coarse phase WORDS (not a %), and a success-
shaped failure
6         // (done but no rallies) is shown, never silently treated as a load.
7         import { useEffect, useState } from "react";
8         import type { ChangeEvent, FormEvent } from "react";
9         import { Trans, useTranslation } from "react-i18next";
10        import { createAsset, getCapabilities } from "../api/client";
11        import type { StorageBackendInfo } from "../api/types";
12        import { useIngestJob } from "../hooks/useIngestJob";
13        import { useUpload } from "../hooks/useUpload";
14        import { classifyError, type IngestError } from "../lib/errors";
15        import { errorText } from "../lib/errorText";
16        import { log, newCorrelationId } from "../lib/log";
17        import { StatusChip, Spinner } from "../StatusChip";
18
19        type Compute = "local" | "cloud";
20        // The three storage-medium choices the picker offers. "bucket" covers s3 OR gcs (the
user pastes a
21        // full scheme'd URI); "drive" is a mounted Google Drive folder
(STUDIO_MEDIA_LIFECYCLE.md P5, D14).
22        type Medium = "local" | "bucket" | "drive";
23
24        interface Props {
25            /** Called with the engine's video_id once a submitted job finishes successfully. */
26            onLoaded: (videoId: string, displayName?: string) => void;
27            /** Lets the shell hide demo-only controls while a real ingest is in flight. */
28            onBusyChange?: (busy: boolean) => void;
29        }
30
31        function formatElapsed(totalSec: number): string {

```

```

32     const sec = Math.max(0, Math.floor(totalSec));
33     const mm = Math.floor(sec / 60);
34     const ss = sec % 60;
35     return `${mm}:${String(ss).padStart(2, "0")}`;
36 }
37
38 function formatEta(totalSec: number | undefined): string | null {
39     if (typeof totalSec !== "number" || !Number.isFinite(totalSec) || totalSec < 0) return
null;
40     const sec = Math.max(0, Math.ceil(totalSec));
41     const hh = Math.floor(sec / 3600);
42     const mm = Math.floor((sec % 3600) / 60);
43     const ss = sec % 60;
44     if (hh > 0) return `${hh}:${String(mm).padStart(2, "0")}:${String(ss).padStart(2,
"0")}`;
45     return `${mm}:${String(ss).padStart(2, "0")}`;
46 }
47
48 function formatUploadSize(bytes: number): string {
49     const b = Math.max(0, Number.isFinite(bytes) ? bytes : 0);
50     if (b === 0) return "0 MB";
51     const gb = b / 1024 ** 3;
52     if (gb >= 1) {
53         const rounded = Math.round(gb * 10) / 10;
54         return `${Number.isInteger(rounded) ? rounded : rounded.toFixed(1)} GB`;
55     }
56     const mb = b / 1024 ** 2;
57     if (mb >= 1) return `${Math.max(1, Math.round(mb))} MB`;
58     return `${Math.max(1, Math.round(b / 1024))} KB`;
59 }
60
61 function sourceLabel(value: string): string | null {
62     const trimmed = value.trim();
63     if (!trimmed) return null;
64     try {
65         const parsed = new URL(trimmed);
66         const tail = parsed.pathname.split(/[\\\/]/).filter(Boolean).pop();
67         return tail ? decodeURIComponent(tail) : trimmed;
68     } catch {
69         return trimmed.split(/[\\\/]/).filter(Boolean).pop() ?? trimmed;
70     }
71 }
72
73 export function IngestPanel({ onLoaded, onBusyChange }: Props) {
74     const { t, i18n } = useTranslation();
75     const [uri, setUri] = useState("");
76     const [activeSource, setActiveSource] = useState<string | null>(null);
77     const [selectedFile, setSelectedFile] = useState<File | null>(null);
78     const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
79     // The UI locale recorded with the submission (PMF analytics + the localized reel
watermark).
80     const locale = i18n.resolvedLanguage ?? i18n.language;
81
82     // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to a
cloud GPU
83     // (capabilities.deployment.cloud_available), let the user choose where DETECT runs.
Default LOCAL
84     // (free, no surprise cost) EXCEPT on a hosted cloud-serving control-plane
(compute_target=cloud_run):
85     // there the control-plane bakes no detector weights, so LOCAL can never run ? default
CLOUD instead
86     // (set from capabilities below), which requires an explicit cost acknowledgement
before it can run.
87     const [cloudAvailable, setCloudAvailable] = useState(false);

```

```

88     const [compute, setCompute] = useState<Compute>("local");
89     const [cloudAck, setCloudAck] = useState(false);
90     // Storage-medium picker (P5): which media THIS box can store into
(capabilities.storage.backends,
91     // usable-only). Absent/local-only ? no picker (today's flow unchanged).
92     const [backends, setBackends] = useState<StorageBackendInfo[]>([]);
93     const [medium, setMedium] = useState<Medium>("local");
94     const [mediumUri, setMediumUri] = useState("");
95     const [storing, setStoring] = useState(false);
96     const [storeError, setStoreError] = useState<IngestError | null>(null);
97     // The engine's max upload size (BYTES), so an oversized file fails INSTANTLY client-
side instead of
98     // uploading into a 413. Absent ? undefined ? no guard (the engine still fail-fasts).
99     const [maxUploadBytes, setMaxUploadBytes] = useState<number | undefined>(undefined);
100    useEffect(() => {
101        let alive = true;
102        getCapabilities()
103            .then((c) => {
104                if (!alive) return;
105                setCloudAvailable(c.deployment?.cloud_available === true);
106                // Hosted cloud-serving control-plane (compute_target=cloud_run) bakes NO
detector weights, so
107                // in-process LOCAL detection there always fails ("weights not found") ? default
the picker to
108                // CLOUD so the SPA never sends compute=local by default on the hosted build. A
local appliance
109                // that merely also offers cloud (compute_target?cloud_run) keeps the free LOCAL
default.
110                if (c.deployment?.cloud_available === true && c.deployment?.compute_target ===
"cloud_run") {
111                    setCompute("cloud");
112                }
113                setBackends(c.storage?.backends ?? []);
114                setMaxUploadBytes(
115                    typeof c.max_upload_mb === "number" && c.max_upload_mb > 0 ? c.max_upload_mb *
1024 * 1024 : undefined,
116                );
117            })
118            .catch((err) => log("warn", "capabilities.unavailable", { error: String(err) }));
119        return () => {
120            alive = false;
121        };
122    }, []);
123    // Only send a choice when the picker is actually shown; otherwise omit it (? server
default).
124    const sendCompute = cloudAvailable ? compute : undefined;
125    // CLOUD selected but the cost note not yet accepted ? block submit (don't silently
run local, and
126    // don't dispatch a paid job without consent).
127    const cloudBlocked = cloudAvailable && compute === "cloud" && !cloudAck;
128
129    // Which medium segments to offer: local always; "bucket" if s3 OR gcs is usable;
"drive" if a Drive
130    // mount is usable. The picker only appears when there's a REAL choice (a non-local
usable backend).
131    const canUse = (id: string) => backends.some((b) => b.id === id && b.usable);
132    const canBucket = canUse("s3") || canUse("gcs");
133    const canDrive = canUse("gdrive");
134    const mediumOptions: Medium[] = ["local", ...(canBucket ? ["bucket" as const] : []),
...(canDrive ? ["drive" as const] : [])];
135    const showMediumPicker = mediumOptions.length > 1;
136    const needsMediumUri = medium !== "local";
137    // A non-local medium needs a destination URI before anything can be stored/submitted.
138    const mediumBlocked = needsMediumUri && mediumUri.trim() === "";
139

```

```

140      // Store the obtained source (an uploaded local path, or a pasted at-rest URI) INTO
the chosen medium,
141      // then process the durable copy. A failed store is loud (surfaced like every other
ingest error).
142      const storeThenProcess = async (src: { uploadedPath?: string; sourceUri?: string },
displayName?: string) => {
143          const cid = newCorrelationId();
144          setStoreError(null);
145          setStoring(true);
146          try {
147              const asset = await createAsset({ ...src, mediumUri: mediumUri.trim() }, cid);
148              log("info", "asset.stored", { cid, video_id: asset.video_id, medium: asset.medium,
copied: asset.copied });
149              submit(asset.stored_uri, locale, sendCompute, displayName ??
sourceLabel(asset.stored_uri) ?? undefined); // index the stored copy
150          } catch (err) {
151              setStoreError(classifyError(err));
152              log("error", "asset.store_failed", { cid, error: String(err) });
153          } finally {
154              setStoring(false);
155          }
156      };
157
158      // On upload completion: local medium ? process the upload directly (today's flow); a
non-local
159      // medium ? store a durable copy into it first, then process. The arrow is recreated
each render
160      // (useUpload reads onCompleteRef.current), so `medium`/`mediumUri`/`sendCompute` are
always fresh.
161      const upload = useUpload((path, filename) => {
162          if (medium === "local") submit(path, locale, sendCompute, filename);
163          else void storeThenProcess({ uploadedPath: path }, filename);
164      }, maxUploadBytes);
165      const busy = ingestBusy || upload.busy || storing;
166      useEffect(() => {
167          onBusyChange?.(busy);
168          return () => {
169              if (busy) onBusyChange?.(false);
170          };
171      }, [busy, onBusyChange]);
172      // Uploading a FILE into a non-local medium needs a destination first. A pasted
cloud/host link is
173      // already an at-rest SOURCE and should process directly regardless of the storage
picker state.
174      const fileInputsDisabled = busy || cloudBlocked || mediumBlocked;
175      const uriInputsDisabled = busy || cloudBlocked;
176
177      const onSubmitUri = (e: FormEvent) => {
178          e.preventDefault();
179          if (uriInputsDisabled) return;
180          // The URI form is always a SOURCE (a bucket URL / host path to index directly). The
medium picker
181          // only controls where a locally picked file is stored before indexing.
182          const displayName = sourceLabel(uri);
183          setSelectedFile(null);
184          setActiveSource(displayName);
185          submit(uri, locale, sendCompute, displayName ?? undefined);
186      };
187      const onFile = (e: ChangeEvent<HTMLInputElement>) => {
188          const file = e.target.files?.[0];
189          // Symmetric with onSubmitUri: never accept a pick while blocked (the disabled input
already
190          // prevents this). Picking only arms the upload; the user starts it explicitly.
191          if (file && !fileInputsDisabled) {
192              upload.reset();

```

```

193         resetIngest();
194         setStoreError(null);
195         setSelectedFile(file);
196         setActiveSource(file.name);
197     }
198     e.target.value = ""; // let re-picking the same file fire change again
199 };
200 const startSelectedFile = () => {
201     if (!selectedFile || fileInputsDisabled) return;
202     setActiveSource(selectedFile.name);
203     upload.start(selectedFile);
204 };
205 const clearSelectedFile = () => {
206     setSelectedFile(null);
207     setActiveSource(null);
208     setStoreError(null);
209     upload.reset();
210     resetIngest();
211 };
212 const cancelAll = () => {
213     upload.cancel();
214     cancelIngest();
215     setSelectedFile(null);
216     setActiveSource(null);
217 };
218 const resetAll = () => {
219     upload.reset();
220     resetIngest();
221     setStoreError(null);
222     setSelectedFile(null);
223     setActiveSource(null);
224 };
225
226 // Shared mapping with the build flow (lib/errorText): an `engine` failure carries the
engine's own
227 // message (a 400 scheme/path/extension detail, a worker error); every other code maps
to a friendly
228 // localized line. Surface whichever half failed (upload or engine) ? they're mutually
exclusive.
229 const errorState =
230     upload.phase.status === "error"
231     ? upload.phase.error
232     : storeError ?? (phase.status === "error" ? phase.error : null);
233 const errorMsg = errorState ? errorText(t, errorState) : null;
234 const visibleSource = upload.phase.status === "uploading" ? upload.phase.filename :
activeSource;
235 const processingName =
236     phase.status === "submitting" || phase.status === "polling" ? phase.displayName :
undefined;
237 // Keep the video's name after a FAILURE too, so the retry screen still names it
(#161) ? including
238 // after a reload, when selectedFile/activeSource are gone and only the restored job
carries the name.
239 const failedName = phase.status === "error" ? phase.displayName : undefined;
240 const currentVideoName = selectedFile?.name ?? visibleSource ?? processingName ??
failedName;
241 const chooseFileLabel =
242     medium === "bucket" ? t("ingest.chooseFileBucket") : medium === "drive" ?
t("ingest.chooseFileDrive") : t("ingest.chooseFile");
243 const uploadAria =
244     medium === "bucket" ? t("ingest.uploadAriaBucket") : medium === "drive" ?
t("ingest.uploadAriaDrive") : t("ingest.uploadAria");
245 const fileHint =
246     medium === "bucket" ? t("ingest.fileHintBucket") : medium === "drive" ?
t("ingest.fileHintDrive") : t("ingest.fileHint");

```

```

247     const uploadEta = upload.phase.status === "uploading" ? formatEta(upload.phase.etaSec)
: null;
248
249     return (
250       <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
251         <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
252         <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
253         {currentVideoName && (
254           <div className="mt-4 max-w-2xl border-l-4 border-green pl-4" data-
testid="ingest-video-title">
255             <p className="text-xs font-semibold uppercase tracking-wide text-
green">{t("ingest.videoTitleLabel")}</p>
256             <p className="mt-1 break-words text-xl font-bold leading-snug text-
ink">{currentVideoName}</p>
257           </div>
258         )}
259
260         {!busy && (
261           <>
262             {/* UPLOAD-MOMENT DISCLOSURE (khelsutra #154): on a cloud-capable server (the
hosted alpha) the
263               footage leaves the device, so testers get an honest, plain-English
disclosure of where it's
264               stored + a link to the full policy. Suppressed on a local appliance
(cloudAvailable=false),
265               where nothing is uploaded. Copy: ALPHA_LAUNCH_REVIEW_2026-07-04.md §4.4. */}
266             {cloudAvailable && (
267               <div
268                 className="mt-4 max-w-2xl rounded-xl border border-amber-300/70 bg-amber-50
p-3 text-sm"
269                 data-testid="upload-disclosure"
270               >
271                 <p className="font-semibold text-ink/80">{t("ingest.disclosure.title")}</p>
272                 <p className="mt-1 text-ink/70">{t("ingest.disclosure.body")}</p>
273                 <a
274                   href="https://khelsutra.guru/privacy"
275                   target="_blank"
276                   rel="noreferrer"
277                   className="mt-2 inline-flex min-h-11 items-center font-medium text-green
underline"
278                 >
279                   {t("ingest.disclosure.link")}
280                 </a>
281               </div>
282             )}
283
284             {/* COMPUTE PICKER (item 3): shown only when this server can dispatch to a cloud
GPU. Default
285               LOCAL; CLOUD reveals a cost note + a required acknowledgement before any
submit can run. */}
286             {cloudAvailable && (
287               <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="compute-picker">
288                 <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-
ink/40">
289                   {t("ingest.compute.legend")}
290                 </legend>
291                 <div className="mt-1 flex flex-wrap gap-2">
292                   {[["local", "cloud"] as const].map((opt) => (
293                     <button
294                       key={opt}
295                       type="button"
296                       aria-pressed={compute === opt}
297                       disabled={busy}

```

```

298         onClick={() => setCompute(opt)}
299         className={
300             "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold
disabled:opacity-40 " +
301             (compute === opt
302             ? "border-green bg-green/10 text-ink"
303             : "border-black/15 text-ink/70 hover:border-green")
304         }
305     >
306         {t(`ingest.compute.${opt}`)}
307     </button>
308 ))}
309 </div>
310 {compute === "cloud" && (
311     <div className="mt-3" data-testid="compute-cost">
312         <p className="text-xs text-ink/60">{t("ingest.compute.cloudCost")}</p>
313         <label className="mt-2 flex items-center gap-2 text-sm text-ink/80">
314             <input
315                 type="checkbox"
316                 checked={cloudAck}
317                 disabled={busy}
318                 onChange={(e) => setCloudAck(e.target.checked)}
319                 className="h-4 w-4"
320             />
321             {t("ingest.compute.ack")}
322         </label>
323         {cloudBlocked && (
324             <p className="mt-1 text-xs text-amber-700" data-testid="compute-ack-
required">
325                 {t("ingest.compute.ackRequired")}
326             </p>
327         )}
328     </div>
329 )}
330 </fieldset>
331 )}
332
333 {/* MEDIUM PICKER (P5, D14): how the user wants to add this video. A non-local
medium still
334     stores a durable MD5-keyed copy before indexing, but the copy uses user-
facing source
335     language because the operator is choosing the route into Studio, not
thinking in backend
336     storage planes. */}
337 {showMediumPicker && (
338     <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="medium-picker">
339         <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-
ink/40">
340             {t("ingest.medium.legend")}
341         </legend>
342         <div className="mt-1 flex flex-wrap gap-2">
343             {mediumOptions.map((opt) => (
344                 <button
345                     key={opt}
346                     type="button"
347                     aria-pressed={medium === opt}
348                     disabled={busy}
349                     onClick={() => {
350                         setMedium(opt);
351                         setStoreError(null);
352                     }}
353                     data-testid={`medium-${opt}`}
354                     className={
355                         "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold

```



```

disabled:opacity-40 " +
356         (medium === opt
357           ? "border-green bg-green/10 text-ink"
358           : "border-black/15 text-ink/70 hover:border-green")
359       }
360   >
361       {t(`storage.${opt}`)}
362   </button>
363   )}}
364 </div>
365 {needsMediumUri && (
366   <div className="mt-3" data-testid="medium-dest">
367     <input
368       type="text"
369       value={mediumUri}
370       onChange={(e) => setMediumUri(e.target.value)}
371       placeholder={medium === "drive" ? t("ingest.medium.drivePlaceholder") :
t("ingest.medium.bucketPlaceholder")}
372       aria-label={t("ingest.medium.destAria")}
373       disabled={busy}
374       className="min-h-11 w-full max-w-md rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
375     />
376     <p className="mt-2 text-xs text-ink/50">{t("ingest.medium.destHint")}</p>
377     {mediumBlocked && (
378       <p className="mt-1 text-xs text-amber-700" data-testid="medium-
required">
379         {t("ingest.medium.destRequired")}
380       </p>
381     )}
382   </div>
383   )}
384 </fieldset>
385 )}
386
387 /* PRIMARY: upload a local file (T4). A native file input, visually a button
(?44px, A4). */
388 <div className="mt-4">
389   <label
390     className={
391       // focus-within makes the visible button show a focus ring when the (sr-
only) input is
392       // focused ? a keyboard user needs a visible focus indicator (WCAG 2.4.7).
393       "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 focus-within:outline-none focus-within:ring-2 focus-
within:ring-green focus-within:ring-offset-2 " +
394       (fileInputsDisabled ? "pointer-events-none opacity-40" : "cursor-pointer")
395     }
396   >
397     {chooseFileLabel}
398     <input
399       type="file"
400       accept=".mp4,.mov,video/mp4,video/quicktime"
401       aria-label={uploadAria}
402       disabled={fileInputsDisabled}
403       onChange={onFile}
404       className="sr-only"
405     />
406   </label>
407   <p className="mt-2 text-xs text-ink/40">{fileHint}</p>
408   {selectedFile && (
409     <div className="mt-3 max-w-2xl" data-testid="selected-file">
410       <p className="text-xs font-medium text-ink/50">
411         {t("ingest.selectedFileReady", { size: formatUploadSize(selectedFile.size)
}})}

```

```

412         </p>
413         <div className="mt-2 flex flex-wrap gap-2">
414             <button
415                 type="button"
416                 onClick={startSelectedFile}
417                 disabled={fileInputsDisabled}
418                 className="inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2
text-sm font-semibold text-white disabled:opacity-40"
419             >
420                 {t("ingest.startSelectedUpload")}
421             </button>
422             <button
423                 type="button"
424                 onClick={clearSelectedFile}
425                 className="inline-flex min-h-11 items-center rounded-xl border border-
black/15 px-4 py-2 text-sm hover:border-green"
426             >
427                 {t("ingest.clearSelectedFile")}
428             </button>
429         </div>
430     </div>
431 )}
432 </div>
433
434     {/* SECONDARY: point at an existing cloud bucket / host path and index it
directly. This stays
435         visible in every medium mode so a gs:// source is never confused with an
upload destination. */}
436     <div className="mt-4 border-t border-black/5 pt-4">
437         <p className="text-xs font-semibold uppercase tracking-wide text-
ink/40">{t("ingest.orLink")}</p>
438         <form onSubmit={onSubmitUri} className="mt-2 flex flex-wrap items-center gap-2">
439             <input
440                 type="text"
441                 value={uri}
442                 onChange={(e) => setUri(e.target.value)}
443                 placeholder={t("ingest.uriPlaceholder")}
444                 aria-label={t("ingest.uriAriaLabel")}
445                 disabled={uriInputsDisabled}
446                 className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
447             />
448             <button
449                 type="submit"
450                 disabled={uriInputsDisabled}
451                 className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
452             >
453                 {phase.status === "submitting" ? t("ingest.submitting") :
t("ingest.submit")}
454             </button>
455         </form>
456         <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
457     </div>
458 </>
459 )}
460
461     {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
462         (A8). Upload shows a determinate % bar; the engine phase is indeterminate
(phase words). */}
463     <div
464         aria-live="polite"
465         className={busy ? "mt-4 border-t border-black/10 pt-4 " : "mt-3 "} + "text-sm"
466         data-testid="ingest-status"

```

```

467     >
468     {upload.phase.status === "uploading" && (
469         <div data-testid="upload-progress">
470             /* Static chip label (not the per-tick %) so the aria-live region announces
once; the live
471                 % rides the progressbar's aria-valuenow + a sighted-only label (A8/T4).
*/
472             <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.uploadingStatus")} />
473             <p className="mt-2 text-xs font-medium text-ink/70" data-testid="ingest-
source">
474                 {t("ingest.sourceFile", { filename: upload.phase.filename })}
475             </p>
476             <p className="mt-1 text-xs text-ink/50" data-testid="upload-meta">
477                 {t("ingest.uploadMeta", {
478                     sent: formatUploadSize(upload.phase.sentBytes),
479                     total: formatUploadSize(upload.phase.sizeBytes),
480                     eta: uploadEta ? t("ingest.uploadEta", { time: uploadEta }) :
t("ingest.uploadEtaEstimating"),
481                 })}
482             </p>
483             <div className="mt-2 flex max-w-sm items-center gap-2">
484                 <div
485                     className="h-1.5 flex-1 overflow-hidden rounded-full bg-black/10"
486                     role="progressbar"
487                     aria-label={t("ingest.uploadProgress")}
488                     aria-valuenow={upload.phase.pct}
489                     aria-valuemin={0}
490                     aria-valuemax={100}
491                 >
492                     <div className="h-full bg-green transition-all" style={{ width:
`$${upload.phase.pct}%` }} />
493                 </div>
494                 <span aria-hidden="true" className="text-xs tabular-nums text-ink/50">
495                     {upload.phase.pct}%
496                 </span>
497             </div>
498             <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
499             <button
500                 type="button"
501                 onClick={cancelAll}
502                 className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
503             >
504                 {t("ingest.cancel")}
505             </button>
506         </div>
507     )}
508     {storing && (
509         <div data-testid="ingest-storing">
510             <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.medium.storing")} />
511             {visibleSource && (
512                 <p className="mt-2 text-xs font-medium text-ink/70" data-testid="ingest-
source">
513                     {t("ingest.sourceFile", { filename: visibleSource })}
514                 </p>
515             )}
516             </div>
517         )}
518     {ingestBusy && (
519         <div>
520             <StatusChip
521                 tone="info"
522                 icon={<Spinner />}

```

```

523         label={
524             t("ingest.working") +
525             (processingName ? ` · ${processingName}` : "") +
526             (phase.status === "polling" && phase.progress
527              ? ` · ${t("ingest.progress", { progress: phase.progress })}`
528              : "")
529         }
530     />
531     {visibleSource && (
532         <p className="mt-2 text-xs font-medium text-ink/70" data-testid="ingest-
source">
533             {t("ingest.sourceFile", { filename: visibleSource })}
534         </p>
535     )}
536     {phase.status === "polling" && (
537         <div className="mt-2 space-y-1 text-xs text-ink/50" data-testid="ingest-
heartbeat">
538             {phase.resumed && <p>{t("ingest.resumeNotice")}</p>}
539             <p>
540                 {t("ingest.elapsed", { elapsed: formatElapsed(phase.elapsedSec) })} ·
{t("ingest.heartbeat")}
541             </p>
542             <p>{t("ingest.reloadHint")}</p>
543         </div>
544     )}
545     {/* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
546     <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
547     <button
548         type="button"
549         onClick={cancelAll}
550         className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
551     >
552         {t("ingest.cancel")}
553     </button>
554 </div>
555 )}
556 {errorMsg && (
557     <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
558         <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
559         {/* Name the failed video right on the retry screen so it's never
lost/untracked (#161). */}
560         {currentVideoName && (
561             <p className="mt-2 text-sm font-semibold text-red-900" data-
testid="ingest-source">
562                 {t("ingest.sourceFile", { filename: currentVideoName })}
563             </p>
564         )}
565         <p className="mt-2">{errorMsg}</p>
566         {/* Oversized upload: a coach may never have trimmed a clip ? offer
concrete, non-technical
567         "how do I trim it?" steps inline (the appliance is air-gapped, so no
links out). Native
568         <details> is keyboard-accessible and needs no JS. */}
569         {errorState?.code === "fileTooLarge" && (
570             <details className="mt-3 text-sm" data-testid="trim-help">
571                 <summary className="cursor-pointer font-medium text-red-900">
572                     {t("errors.fileTooLargeHelp.summary")}
573                 </summary>
574                 <ul className="mt-2 list-disc space-y-1 pl-5 text-red-900/80">
575                     <li>
576                         <Trans i18nKey="errors.fileTooLargeHelp.phone" components={{ b:
<strong /> }} />

```

```

577             </li>
578             <li>
579                 <Trans i18nKey="errors.fileTooLargeHelp.computer" components={{ b:
<strong /> }} />
580             </li>
581             <li>
582                 <Trans i18nKey="errors.fileTooLargeHelp.advanced" components={{ b:
<strong /> }} />
583             </li>
584         </ul>
585     </details>
586 )}
587 <button
588     type="button"
589     onClick={resetAll}
590     className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
591     >
592         {t("ingest.retry")}
593     </button>
594 </div>
595 )}
596 </div>
597 </section>
598 );
599 }
600

```

12. assistant (2026-07-07T15:08:32.606Z)

Now let me read the full orchestration file and the backend/SPA supporting files in parallel.

13. user (2026-07-07T15:08:35.403Z)

```

1     """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3     The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4     Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5     and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6     Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8     SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9     monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10    ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11    the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12    bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13    ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14    This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15    Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16    ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17    Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18    objects and need no routing.
19    """

```

```

20
21 import csv
22 import json
23 import logging
24 import math
25 import os
26 import re
27 import tempfile
28 import threading
29 from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30 from urllib.parse import quote
31
32 from backend import storage
33 from backend.config import AppConfig, StitchingConfig
34 from backend.pipeline.segmenters.base import segmenter_registry
35 from backend.pipeline.stitcher.watermark_i18n import localize_watermark
36 from backend.pipeline.validators.base import registry
37 from backend.results import Failure
38 from backend.utils.paths import output_base, safe_output_filename
39
40 if (
41     TYPE_CHECKING
42 ): # runtime import would be circular (main imports us for the re-export)
43     from backend.main import ProcessRequest # noqa: F401
44
45 logger = logging.getLogger(__name__)
46
47 # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
48 compile stitch ?
49 # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
50 dispatch/poll) never
51 # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
52 Gemini is
53 # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
54 remote jobs overlap
55 # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
56 deadlock; when the
57 # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
58 _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
59
60 def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
61     """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
62
63     On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
64     fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
65     and
66     keep the prior good results intact ? a failed/empty re-run never destroys prior
67     data.
68     """
69     if segments:
70         db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
71     else:
72         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
73
74 def _validation_failure_message(failures: List[Any]) -> str:
75     details = []
76     for failure in failures:
77         name = getattr(failure, "validator_name", None) or "validation"
78         message = (getattr(failure, "message", None) or "").strip()
79         details.append(f"{name}: {message}" if message else str(name))
80     if not details:
81         return "Video failed validation checks."
82     return "Video failed validation: " + "; ".join(details)

```

[illegible]

```

134         break
135     try:
136         start, end = float(row["start"]), float(row["end"])
137     except (KeyError, TypeError, ValueError):
138         continue
139     if not (math.isfinite(start) and math.isfinite(end)):
140         continue # reject inf/nan times rather than persist a junk rally
141     meta: Dict[str, Any] = {
142         "source": "cloud_run",
143         "ending_reason": (row.get("ending_reason") or "").strip(),
144     }
145     sc = (row.get("shot_count") or "").strip()
146     if sc:
147         try:
148             meta["shot_count"] = int(
149                 float(sc)
150             ) # OverflowError on 'inf'/'1e400'
151         except (ValueError, OverflowError):
152             meta["shot_count"] = sc
153     db.add_play_segment(video_id, start, end, metadata=meta)
154     n += 1
155     return n
156
157
158     # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
159     # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
160     # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
161     # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
162     # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
163     _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
164
165
166     def _local_detector_weights_present(cfg, seg_name: str) -> bool:
167         """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
168         on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
169         and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
170         otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
171         A non-weights segmenter returns True (nothing local to check)."""
172         name = (seg_name or "").strip().lower()
173         if name not in _WEIGHTS_BACKED_SEGMENTERS:
174             return True
175         idx = getattr(cfg, "indexing", None)
176         substrate = (
177             getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
178         ).strip().lower()
179         if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
180             path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
181         else: # wasb_hybrid, or fusion on the (default) WASB substrate
182             path = (
183                 os.environ.get("WASB_WEIGHTS")
184                 or getattr(getattr(idx, "wasb", None), "weights_path", "")
185                 or ""
186             )
187         return bool(path) and os.path.exists(os.path.expanduser(path))

```



```

188
189
190     def _cloud_available(cfg) -> bool:
191         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
192         3).
193         True when the control plane is wired for Cloud Run: either the configured default
194         target is
195         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
196         coordinates
197         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
198         (``storage.output_root``)
199         are present so a forced per-request override can actually dispatch + collect a
200         result.
201         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
202         would work."""
203         deployment = getattr(cfg, "deployment", None)
204         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
205             return True
206         has_job = bool(
207             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
208         )
209         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
210         return bool(has_job and out_root)
211
212     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
213         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
214         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
215         comes
216         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
217         "starting"
218         (queue/cold-start), so the very first tick already shows movement."""
219         minutes = int(elapsed_sec) // 60
220         if minutes < 1:
221             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
222         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
223
224     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
225         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
226         gs://
227         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
228         keep
229         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
230         no
231         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
232         if getattr(input_ref, "backend", "") != "gs":
233             return None
234         sc = _storage_settings(cfg)
235         if not sc:
236             return None
237         try:
238             st = storage.get_storage("gs", config=sc).stat(input_ref)
239             return getattr(st, "md5", None) or None
240         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
241             logger.info(
242                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
243                 getattr(input_ref, "uri", input_ref),
244                 exc_info=True,
245             )
246         return None

```

```

242     def _cached_process_payload(
243         video_id: str, validation_results, segments, requested_compute
244     ) -> Dict[str, Any]:
245         """The /api/process result for a video whose detection already exists ? ONE shape
for
246         both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
247         return {
248             "video_id": video_id,
249             "status": "success",
250             "message": (
251                 f"Reused {len(segments)} cached play segments. "
252                 "Pass force=true to reprocess this video."
253             ),
254             "results": validation_results or [],
255             "play_segments": segments,
256             "cached": True,
257             "compute": {"path": "cached", "requested": requested_compute},
258         }
259
260
261     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
262         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
263         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
264         ref = storage.parse_uri(uri)
265         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
266             local = storage.fetch(
267                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
268             )
269             with open(local, encoding="utf-8") as f:
270                 loaded = json.load(f)
271             return loaded if isinstance(loaded, dict) else None
272
273
274     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
275     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
276     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
277     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
278     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
279     _REHYDRATE_LOCKS_GUARD = threading.Lock()
280
281
282     def _rehydrate_lock(video_id: str) -> threading.Lock:
283         with _REHYDRATE_LOCKS_GUARD:
284             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
285
286
287     def probe_process_cache(db, cfg, video_id: str) -> bool:
288         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
289
290         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
291         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
292         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
293         errors report False (? the normal dispatch path with its guards)."""
294         try:
295             existing = db.get_video(video_id)
296             if (
297                 existing
298                 and existing.get("status") == "processed"

```

```

299         and db.query_play_segments(video_id, {}))
300     ):
301         return True
302     sc = _storage_settings(cfg)
303     if not sc or (sc.get("backend") or "local") == "local":
304         return False
305     output_root = (sc.get("output_root") or "").rstrip("/")
306     if not output_root:
307         return False
308     report_uri = f"{output_root}/outputs/{video_id}/report.json"
309     if not storage.exists(report_uri, config=sc):
310         return False
311     report = _read_bucket_json(report_uri, sc)
312     return bool(report and report.get("status") == "success")
313 except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
314     logger.warning(
315         "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
316     )
317     return False
318
319
320 def cached_process_result(
321     db, cfg, video_id: str, video_path: str, requested_compute=None
322 ) -> Optional[Dict[str, Any]]:
323     """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
324     the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
325     re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
326     source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
327     completed work found (or the check failed) ? the caller dispatches normally. The
bucket
328     branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
329     partial earlier ingest is replaced, never double-counted.
330
331     Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
332     per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
333     in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
334     job before invoking (writes happen post-claim)."""
335     try:
336         with _rehydrate_lock(video_id):
337             existing = db.get_video(video_id)
338             if existing and existing.get("status") == "processed":
339                 segs = db.query_play_segments(video_id, {})
340                 if segs:
341                     return _cached_process_payload(
342                         video_id,
343                         existing.get("validation_results", []),
344                         segs,
345                         requested_compute,
346                     )
347             sc = _storage_settings(cfg)
348             if not sc or (sc.get("backend") or "local") == "local":
349                 return None
350             output_root = (sc.get("output_root") or "").rstrip("/")
351             if not output_root:
352                 return None
353             outputs_uri = f"{output_root}/outputs/{video_id}"
354             report_uri = f"{outputs_uri}/report.json"
355             if not storage.exists(report_uri, config=sc):
356                 return None

```

```

357     report = _read_bucket_json(report_uri, sc)
358     if not report or report.get("status") != "success":
359         return None
360     if not video_path:
361         # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
362         # carries the worker's --video-uri (P3); older reports fall back to the
363         # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
364         # recover, and the next same-md5 submit re-binds the truthful path via
365         # add_video's upsert.
366         video_path = str(report.get("video_uri") or "") or _first_object_under(
367             f"{output_root}/videos/{video_id}/", sc
368         )
369     play_wm, cand_wm = db.segment_watermarks(video_id)
370     db.add_video(video_id, video_path, "processed", [])
371     n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
372     if n <= 0:
373         return None
374     segments = [
375         s
376         for s in db.query_play_segments(video_id, {})
377         if int(s.get("id", 0)) > play_wm
378     ]
379     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
380     if not segments:
381         return None
382     logger.info(
383         "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
384         n,
385         video_id,
386         outputs_uri,
387     )
388     return _cached_process_payload(video_id, [], segments, requested_compute)
389 except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
390     logger.warning(
391         "[API] submit-time cache check failed for %s ? dispatching normally",
392         video_id,
393         exc_info=True,
394     )
395     return None
396
397
398 def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
399     """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
400     try:
401         return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
402     except Exception: # noqa: BLE001 ? recovery nicety, never a failure
403         return ""
404
405
406 _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
407
408
409 def read_path_rehydrate(db, cfg, video_id: str) -> bool:
410     """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
411     nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/` ` ?
so
412     ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
DB
413     has the video's rows now (re-query). Non-md5 ids return False without touching the
414     bucket; every failure degrades to False (the read just reports what the DB has)."""
415     vid = (video_id or "").strip().lower()
416     if not _MD5_HEX_RE.fullmatch(vid):

```

```

417         return False
418     if not probe_process_cache(db, cfg, vid):
419         return False
420     return cached_process_result(db, cfg, vid, "") is not None
421
422
423     def _maybe_dispatch_remote(
424         cfg,
425         db,
426         req: "ProcessRequest",
427         video_id: str,
428         seg_name: str,
429         val_results,
430         play_wm: int,
431         cand_wm: int,
432         notify,
433     ) -> Optional[Tuple[Dict[str, Any], bool]]:
434         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
435         ``deployment.compute_target
436         == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
437         intervals into
438         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
439
440         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
441         the cloud job
442         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
443         observability report
444         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
445         process segmenter
446         (**DEFAULT-OFF*: ``get_compute_backend`` returns ``None`` for every
447         non-``cloud_run`` target, so
448         the in-process path stays byte-identical)."""
449         from backend.compute import ComputeJobSpec, get_compute_backend
450
451         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
452             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
453             (id > play_wm);
454             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
455             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
456             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
457             only when the
458             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
459             rc / ingest
460             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
461             records the
462             # intended backend, `requested` the picker choice, `dispatched` whether the
463             remote job ran.
464             return (
465                 {
466                     "video_id": video_id,
467                     "status": "failed",
468                     "message": msg,
469                     "results": [r.model_dump() for r in val_results],
470                     "play_segments": [],
471                     "compute": {
472                         "path": "cloud_run" if ran else "not_run",
473                         "requested": req.compute,
474                         "target": "cloud_run",
475                         "dispatched": ran,
476                     },
477                 },
478                 ran,
479             )
480
481     # SPA compute-picker (item 3): a per-request choice overrides the server's

```

```

configured default.
471     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
472     # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
473     choice = (req.compute or "").strip().lower()
474     if choice and choice not in ("local", "cloud"):
475         logger.warning(
476             "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
477             req.compute,
478         )
479     choice = ""
480     if choice == "local":
481         # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
482         # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
483         # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
484         # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
485         # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
486         # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
487         # below exactly as before (the weights-present check is the escape hatch).
488         if (
489             (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
490             and _cloud_available(cfg)
491             and not _local_detector_weights_present(cfg, seg_name)
492         ):
493             return _failed(
494                 f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
495                 "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
496                 False,
497             )
498             return None # explicit "run on my machine" ? in-process regardless of the
server's target
499             if choice == "cloud":
500                 if not _cloud_available(cfg):
501                     return _failed(
502                         "cloud-serving was requested but this server isn't configured for it "
503                         "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
504                         False,
505                     )
506                     compute = get_compute_backend(cfg, target_override="cloud_run")
507             else:
508                 compute = get_compute_backend(
509                     cfg
510                 ) # server default (default-OFF unless cloud_run)
511             if compute is None:
512                 return None # default-OFF ? run the in-process segmenter below (unchanged)
513
514             storage_cfg = getattr(cfg, "storage", None)
515
516             # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
517             try:
518                 input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
519             except ValueError as e:
520                 return _failed(

```

```

521         f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
522     )
523     if input_ref.backend == "local":
524         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
525         # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
526         # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
527         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
528         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
529         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
530         # default, while a bare/local-looking root can't stage no matter what
input_backend says
531         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
532         # gives us nothing to build the staged URI under).
533         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
534         try:
535             root_backend = (
536                 storage.parse_uri(input_root).backend if input_root else "local"
537             )
538         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
539             root_backend = "local"
540         if root_backend == "local":
541             return _failed(
542                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
543                 "stage local uploads into; a path local to this machine can't be read by
the Cloud "
544                 "Run job.",
545                 False,
546             )
547         local_src = input_ref.key # resolved local path of the upload
548         staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
549         try:
550             notify("staging upload to bucket")
551             # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
552             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
553             storage.put(
554                 local_src,
555                 staged_uri,
556                 config=(
557                     storage_cfg.model_dump() if storage_cfg is not None else None
558                 ),
559             )
560         except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
561             logger.error(
562                 "[API] cloud-serving: failed to stage %s -> %s: %s",
563                 local_src,
564                 staged_uri,
565                 e,
566                 exc_info=True,
567             )
568             return _failed(
569                 f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",

```

```

570         False,
571     )
572     # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
573     # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
574     # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
575     req.video_path = staged_uri
576     input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
577     db.add_video(
578         video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
579     )
580     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
581     if not out_root:
582         return _failed(
583             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
584             "? that's where the Cloud Run job writes the result.",
585             False,
586         )
587
588     notify("dispatching (cloud_run)")
589     logger.info(
590         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
591         req.video_path,
592         out_root,
593     )
594     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
595     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
596     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
597     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
598     overrides = []
599     idx_cfg = getattr(cfg, "indexing", None)
600     sk = getattr(idx_cfg, "skip_ai_handoff", None)
601     if sk is not None:
602         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
603     # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
604     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
605     # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
606     # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
607     # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
608     # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
609     wasb_cfg = getattr(idx_cfg, "wasb", None)
610     if wasb_cfg is not None:
611         decode_backend = getattr(wasb_cfg, "decode_backend", None)
612         if decode_backend:
613             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
614         batch_size = getattr(wasb_cfg, "batch_size", None)
615         if batch_size:
616             overrides.append(f"indexing.wasb.batch_size={batch_size}")
617         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
618         if prefetch:
619             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
620     spec = ComputeJobSpec(

```



```

621         video_uri=req.video_path,
622         out_uri=out_root,
623         segmenter=seg_name,
624         config_overrides=tuple(overrides),
625     )
626     try:
627         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
628         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
629         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
630         # also gives the SPA a stall signal to time out on, instead of a blind wall
631         result = compute.run_infer(
632             spec,
633             on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
634         )
635     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
636         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
637         return _failed(f"cloud dispatch error: {e}", True)
638     if result.returncode != 0:
639         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
640     if result.video_id and result.video_id != video_id:
641         # The worker re-hashes the bucket bytes; a divergence means the input changed
642         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
643         # diagnosable.
644         logger.warning(
645             "[API] cloud video_id %s != local %s ? input bytes differ between the API
646             hash "
647             "and the worker hash; ingesting under the local id.",
648             result.video_id,
649             video_id,
650         )
651     notify("ingesting (cloud_run)")
652     storage_dict = (
653         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
654         can't narrow
655         if hasattr(storage_cfg, "model_dump")
656         else {}
657     )
658     try:
659         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
660     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
661         # 500 or leak rows
662         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
663         return _failed(
664             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
665             True,
666         )
667     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
668     # drops the old ones.
669     segments = [
670         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
671     ]
672     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
673     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
674     # is the real
675     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
676     # <execution>).
677     provenance = {
678         "path": "cloud_run",
679         "requested": req.compute,

```

```

675         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
676         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
677         "execution": result.execution,
678         "outputs_uri": result.outputs_uri,
679     }
680     logger.info(
681         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
682         result.execution,
683         provenance["job"],
684         provenance["region"],
685         result.outputs_uri,
686         video_id,
687     )
688     return (
689         {
690             "video_id": video_id,
691             "status": "success",
692             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
693             "results": [r.model_dump() for r in val_results],
694             "play_segments": segments,
695             "compute": provenance,
696         },
697         True,
698     )
699
700
701 def _execute_processing(
702     config=None, db=None, req=None, progress=None
703 ) -> Dict[str, Any]:
704     """The full hash ? validate ? segment ? report pipeline for one video.
705
706     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
707     module-level ``global_config``/``db_instance`` (backward-compatible with existing
708     tests that pass only ``req``). Route handlers always pass explicit params; older
709     test code that calls ``_execute_processing(req)`` still works.
710
711     ``progress`` is an optional callable(stage: str) used to surface coarse job
712     progress.
713
714     Raises on unexpected internal errors ? the caller records those as a job failure
715     """
716     import backend.main as _main # call-time seam: main's globals/model stay patchable
717
718     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
719     # shift positional args ? req arrives in position 0 (config), progress in position 1
720     (db).
721
722     if config is not None and not isinstance(config, AppConfig):
723         # Old signature: _execute_processing(req, progress=None)
724         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
725             req = db
726             db = None
727         elif req is None and isinstance(config, _main.ProcessRequest):
728             req = config
729             config = None
730         # Resolve config/db from module globals when not passed explicitly
731         if config is None:
732             config = _main.global_config
733         if db is None:
734             db = _main.db_instance
735
736     notify = progress or (lambda stage: None)
737
738     notify("hashing")
739     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged

```

```

(identity ?
736     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
737     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
738     # consumers (hash / validators / segmenter) use the resolved local path.
739     local_video = ""
740     try:
741         local_video = storage.acquire_local_input(
742             req.video_path, getattr(config, "storage", None)
743         )
744         video_id = _main.compute_video_id(local_video)
745     except Exception:
746         storage.cleanup_acquired_local_input(
747             req.video_path, local_video, getattr(config, "storage", None)
748         )
749         raise
750     try:
751         existing_video = db.get_video(video_id)
752     except Exception:
753         storage.cleanup_acquired_local_input(
754             req.video_path, local_video, getattr(config, "storage", None)
755         )
756         raise
757     was_reingest = existing_video is not None
758
759     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
760     default_seg = getattr(
761         getattr(config, "indexing", None), "default_segmenter", "motion"
762     )
763     seg_name = req.segmenter or default_seg
764     segmenter_actual = None
765     segmentation_ran = False
766     val_results: List[Any] = []
767     val_context: Dict[str, Any] = {}
768     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
769     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
770     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
771     compute_actual: Optional[str] = None
772     response: Optional[Dict[str, Any]] = None
773     try:
774         if (
775             existing_video
776             and existing_video.get("status") == "processed"
777             and not req.force
778         ):
779             cached_segments = db.query_play_segments(video_id, {})
780             if cached_segments:
781                 logger.info(
782                     "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
783                     video_id,
784                 )
785             response = _cached_process_payload(
786                 video_id,
787                 existing_video.get("validation_results", []),
788                 cached_segments,
789                 getattr(req, "compute", None),
790             )
791             return response
792
793     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for

```

```

the report)
794     notify("validating")
795     logger.info(f"Running validations for {req.video_path}...")
796     val_results, val_context = registry.run_all_with_context(local_video, config)
797     failures = [r for r in val_results if not r.passed]
798
799     if failures:
800         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
801         # status; it no longer destroys the video's prior good segments.
802         db.add_video(
803             video_id,
804             req.video_path,
805             "failed",
806             [r.model_dump() for r in val_results],
807         )
808         response = {
809             "video_id": video_id,
810             "status": "failed",
811             "message": _validation_failure_message(failures),
812             "results": [r.model_dump() for r in val_results],
813         }
814         return response
815
816     # 2. Run segmenter & indexer
817     logger.info("[API] Video passed checks. Segmenting and indexing...")
818     db.add_video(
819         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
820     )
821
822     segmenter_cls = segmenter_registry.get(seg_name)
823     if not segmenter_cls:
824         # Submit-time validation already 400s unknown names; this is the defensive
825         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
826         available = list(segmenter_registry.list_available().keys())
827         response = {
828             "video_id": video_id,
829             "status": "failed",
830             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
831             "results": [r.model_dump() for r in val_results],
832             "play_segments": [],
833         }
834         return response
835
836     logger.info(f"[API] Using segmenter: {seg_name}")
837     notify(f"segmenting ({seg_name})")
838     segmenter_actual = seg_name
839     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
840     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
841     play_wm, cand_wm = db.segment_watermarks(video_id)
842     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
843     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
844     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
845     # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
846     # they're dropped on the cloud path in this slice ? a documented follow-up.)
847     remote = _maybe_dispatch_remote(
848         config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
849     )

```

```

850         if remote is not None:
851             response, segmentation_ran = (
852                 remote # `ran` = did the cloud job actually execute (report)
853             )
854             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
855         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
856         compute_actual = "in_process"
857         segmenter = segmenter_cls(db, config)
858         try:
859             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
860             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
861             with _LOCAL_COMPUTE_LANE:
862                 result = segmenter.process_video(
863                     video_id, local_video, req.player_pool, req.max_frames
864                 )
865             except Exception:
866                 # A raising segmenter must not leave half-written rows: restore the pre-run
867                 # state (same destroy-after-confirm contract as the Failure branch), then
let
868                 # the job worker record the failure.
869                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
870                 raise
871             segmentation_ran = True
872
873             if isinstance(result, Failure):
874                 logger.error(f"[API] Segmenter failed: {result.message}")
875                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
876                 response = {
877                     "video_id": video_id,
878                     "status": "failed",
879                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
880                     "results": [r.model_dump() for r in val_results],
881                     "play_segments": [],
882                 }
883                 return response
884
885             segments = result.value if hasattr(result, "value") else result
886             notify("finalizing")
887             _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
888
889             response = {
890                 "video_id": video_id,
891                 "status": "success",
892                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
.value
893                 "results": [r.model_dump() for r in val_results],
894                 "play_segments": segments,
895             }
896             return response
897         finally:
898             # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
899             # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
900             # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
901             # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
902             if isinstance(response, dict):
903                 response.setdefault(

```

```

904         "compute",
905         {
906             "path": compute_actual or "not_run",
907             "requested": getattr(req, "compute", None),
908         },
909     )
910     prov = response.get("compute", {}) if isinstance(response, dict) else {}
911     logger.info(
912         "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
913         video_id,
914         getattr(req, "compute", None),
915         prov.get("path"),
916         prov.get("dispatched"),
917         (response or {}).get("status") if isinstance(response, dict) else None,
918     )
919     # Always emit the per-video observability report (next to the video, or to
920     # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
921     # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
922     # doesn't surface arbitrary run signals.)
923     paths = _main.emit_indexer_report(
924         db=db,
925         video_id=video_id,
926         # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
927         # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
928         # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
929         # still preserved in DB provenance via db.add_video.
930         video_path=local_video,
931         config=_report_config_for_input(config, req.video_path),
932         val_results=val_results,
933         val_context=val_context,
934         segmenter_requested=req.segmenter,
935         segmenter_actual=segmenter_actual,
936         was_reingest=was_reingest,
937         segmentation_ran=segmentation_ran,
938     )
939     if paths and isinstance(response, dict):
940         response["reports"] = paths
941     storage.cleanup_acquired_local_input(
942         req.video_path, local_video, getattr(config, "storage", None)
943     )
944
945
946     def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
947         """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
948
949         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
950         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
951         to a no-op rather than blocking a job it cannot measure."""
952         try:
953             sc = (
954                 storage_config.model_dump()
955                 if hasattr(storage_config, "model_dump")
956                 else (storage_config or {})
957             )
958             backend = storage.get_storage(input_ref.backend, config=sc)
959             return int(backend.stat(input_ref).size)
960         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
not block

```

```

961         logger.debug(
962             "free-space: could not stat remote source %s: %s", input_ref.uri, e
963         )
964         return None
965
966
967     class _CompileError(Exception):
968         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
969         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
970
971         def __init__(self, status_code: int, detail: str):
972             super().__init__(detail)
973             self.status_code = status_code
974             self.detail = detail
975
976
977     def _resolve_compile(
978         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
979     ):
980         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
981         row could change between submit and run). Verifies the video exists, the requested
segments match
982         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
983         so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
984         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
985         video = db.get_video(video_id)
986         if not video:
987             raise _CompileError(404, "Indexed video record not found.")
988         # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
989         try:
990             out_name = safe_output_filename(output_filename)
991         except ValueError as e:
992             raise _CompileError(400, str(e)) from e
993
994         all_segments = db.query_play_segments(video_id, {})
995         matched = [s for s in all_segments if s["id"] in segment_ids]
996         if not matched:
997             raise _CompileError(400, "No matching play segments found to stitch.")
998
999         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1000         kept = []
1001         for s in matched:
1002             meta = s.get("metadata") or {}
1003             sc = meta.get("shot_count") if isinstance(meta, dict) else None
1004             try:
1005                 if sc is not None and int(sc) < stitching.min_shot_count:
1006                     continue
1007             except (TypeError, ValueError):
1008                 pass # unparseable count -> treat as unknown, keep
1009             kept.append(s)
1010         if len(kept) < len(matched):
1011             logger.info(
1012                 "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
1013                 stitching.min_shot_count,
1014                 len(matched) - len(kept),
1015                 len(matched),
1016             )

```

```

1017         if not kept:
1018             raise _CompileError(
1019                 400,
1020                 f"All {len(matched)} matching segments fall below "
1021                 f"stitching.min_shot_count={stitching.min_shot_count}.",
1022             )
1023         return video, kept, out_name
1024
1025
1026     def reel_object_uri(cfg, video_id: str, name: str) -> str:
1027         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
1028         (#463), or "" when
1029         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
1030         compile
1031         mirror-upload and the /api/highlights redirect:
1032         ``<output_root>/highlights/<video_id>/<name>``. """
1033         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1034         if not root or "://" not in root:
1035             return ""
1036         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1037
1038
1039     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1040         """The active storage settings dict (per-backend config for the storage layer ? S3
1041         endpoint/region,
1042         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
1043         default backend
1044         isn't silently constructed as a fresh default. """
1045         sc = getattr(cfg, "storage", None)
1046         return sc.model_dump() if sc is not None else None
1047
1048
1049     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1050         """A short-lived signed URL for a reel mirrored to the output bucket, so
1051         /api/highlights can
1052         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
1053         ephemeral local
1054         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
1055         config for BOTH
1056         the existence check and the signing (extracted here to keep backend/main.py lean ?
1057         P23/P24). """
1058         reel_uri = reel_object_uri(cfg, video_id, name)
1059         if not reel_uri:
1060             return None
1061         settings = _storage_settings(cfg)
1062         if storage.exists(reel_uri, config=settings):
1063             return storage.signed_url(reel_uri, config=settings, method="GET")
1064         return None
1065
1066
1067     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1068         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
1069         stitch the reel,
1070         and return the {status, filepath, download_url} result the SPA polls for. Returns a
1071         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
1072         stitch error)
1073         so the SPA always gets a clean verdict instead of a worker-crash error. """
1074         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
1075         backend.main
1076
1077         notify = progress or (lambda stage: None)
1078         params = json.loads(job.get("params") or "{}")
1079         video_id = job.get("video_id") or ""
1080         segment_ids = params.get("segment_ids") or []
1081         locale = params.get("locale")

```



```

1070
1071     notify("resolving")
1072     try:
1073         video, kept, out_name = _resolve_compile(
1074             db, cfg, video_id, segment_ids, params.get("output_filename", "")
1075         )
1076     except _CompileError as e:
1077         return {"status": "failed", "message": e.detail, "video_id": video_id}
1078
1079     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1080     output_dir = (
1081         getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1082     )
1083     # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1084     # unknown/absent locale keeps the configured default unchanged.
1085     localized_watermark = localize_watermark(stitching.watermark, locale)
1086     compiler = _main.VideoCompiler(
1087         output_dir,
1088         begin_padding=stitching.begin_padding,
1089         end_padding=stitching.end_padding,
1090         watermark=localized_watermark,
1091     )
1092     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1093
1094     notify("stitching")
1095     local_src = ""
1096     try:
1097         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1098         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1099         # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1100         local_src = storage.acquire_local_input(
1101             video["filepath"], getattr(cfg, "storage", None)
1102         )
1103         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1104         # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1105         with _LOCAL_COMPUTE_LANE:
1106             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1107     except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1108         logger.error(
1109             "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1110         )
1111         return {
1112             "status": "failed",
1113             "message": f"Highlight stitching failed: {e}",
1114             "video_id": video_id,
1115         }
1116     finally:
1117         if local_src:
1118             storage.cleanup_acquired_local_input(
1119                 video["filepath"], local_src, getattr(cfg, "storage", None)
1120             )
1121
1122     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1123     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1124     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint

```

```

1125         # still falls back to the local file on the same instance).
1126         reel_uri = reel_object_uri(cfg, video_id, out_name)
1127         if reel_uri:
1128             try:
1129                 notify("uploading reel")
1130                 # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1131                 # project/client) is used, not a fresh default.
1132                 storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1133             except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1134                 logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1135
1136         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1137         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1138         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1139         notify("finished")
1140         return {
1141             "status": "success",
1142             "filepath": out_path,
1143             "download_url": download_url,
1144             "video_id": video_id,
1145         }
1146

```

14. assistant (2026-07-07T15:08:42.343Z)

Now let me examine the capabilities contract on both sides: the backend handler and the SPA types/client.

15. user (2026-07-07T15:08:44.637Z)

```

1      // Types mirror the engine's verified contract (badminton-highlight-indexer).
2      // Stable anchors: backend/infrastructure/database.py play_segments/candidate_segments,
3      // and engine routes/symbols for query, compile, highlights, config, videos, jobs, and
process.
4
5      export interface RallyMetadata {
6          /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7          shot_count?: number;
8          /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9          shot_count_confidence?: string;
10         detector?: string;
11         [key: string]: unknown;
12     }
13
14     export interface RallySegment {
15         /** The integer the UI selects and passes to /api/compile as segment_ids. */
16         id: number;
17         start_time: number;
18         end_time: number;
19         duration: number;
20         /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21         server: string | null;
22         /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23         receiver: string | null;
24         metadata: RallyMetadata;
25         /** motion-only fabricated sub-events ? DO NOT surface. */
26         events?: unknown[];
27     }
28
29     export interface QueryResponse {

```

```

30     play_segments: RallySegment[];
31 }
32
33 /** One row of the indexed-video library. GET /api/videos [engine list_videos route]
returns the most
34 * recent first so the console survives a reload. Every field but `video_id` is
optional/defensive. */
35 export interface VideoSummary {
36     /** The MD5 join key (the contract field). Optional defensively ? an older/regressed
engine may
37     * send only the raw `id`; read it through `lib/video.ts::videoId`, never directly.
*/
38     video_id?: string;
39     /** Raw DB fields an engine may send instead of the enriched contract (resilience
fallbacks). */
40     id?: string;
41     filepath?: string | null;
42     match_name?: string | null;
43     local_path?: string | null;
44     fps?: number | null;
45     resolution?: string | null;
46     created_at?: string | null;
47 }
48
49 export interface VideosResponse {
50     videos: VideoSummary[];
51 }
52
53 /** POST /api/annotations ? persist finished rally labels for a video as the annotator
CSV (P8).
54 * The engine writes the CSV verbatim (byte-compatible with the pipeline), keyed by the
file MD5. */
55 export interface AnnotationsResponse {
56     status: string;
57     num_rows?: number;
58     csv_uri?: string;
59     message?: string;
60 }
61
62 export interface CompileResponse {
63     status: string;
64     /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
65     filepath: string;
66 }
67
68 /** The terminal RESULT of an async reel compile (#329): the job's `result` payload,
surfaced by
69 * GET /api/jobs/{id} once the stitch finishes. `status` is the pipeline verdict
("success" |
70 * "failed"); a "failed" result carries a `message`. The SPA builds the download URL
with
71 * highlightUrl() (split-origin-safe), but the engine also returns a same-origin
`download_url`. */
72 export interface CompileResult {
73     status: string;
74     filepath?: string;
75     download_url?: string;
76     message?: string;
77     video_id?: string;
78 }
79
80 export interface EngineConfig {
81     stitching?: {
82         /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
*/

```

```

83         min_shot_count?: number;
84         [key: string]: unknown;
85     };
86     [key: string]: unknown;
87 }
88
89 /** GET /api/capabilities [backend/main.py] ? an HONEST probe of what THIS box can do,
rendered by
90 * the first-run wizard (PACKAGING_PLAN.md P2c). Presence-bools only (never the Gemini
key value).
91 * Every field optional/defensive: a missing field degrades the wizard gracefully. */
92 export interface EngineCapabilities {
93     api_version?: number;
94     engine_version?: string;
95     segmenters?: string[];
96     default_segmenter?: string;
97     ffmpeg?: { ffmpeg?: boolean; ffprobe?: boolean; ffmpeg_path?: string; ffprobe_path?:
string };
98     gpu?: { nvidia_smi_present?: boolean };
99     /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step.
*/
100     gemini_key_present?: boolean;
101     wasb_weights_present?: boolean;
102     /** The largest upload THIS box accepts, in MB (engine `security.max_upload_mb`,
default 4096).
103     * /upload/init already 413s an oversized file BEFORE any part transfers; the SPA
consumes this to
104     * reject it INSTANTLY client-side (no wasted round-trip). Absent on older engines ?
the guard is
105     * skipped and the engine's server-side fail-fast still applies. */
106     max_upload_mb?: number;
107     /** `cloud_available` = can THIS server satisfy a per-request compute="cloud" dispatch
(a Cloud Run
108     * GPU Job)? The ingest compute-picker shows the "in the cloud" option only when
true. */
109     deployment?: { profile?: string; compute_target?: string; cloud_available?: boolean };
110     /** Which storage media THIS box can store INTO (P3). The medium picker renders only
`usable`
111     * backends; `free_gb` is `null` for a cloud bucket (no readable capacity). Absent on
older engines
112     * ? the SPA degrades to local-only (no picker). */
113     storage?: StorageCapabilities;
114     [key: string]: unknown;
115 }
116
117 /** One storable medium reported by GET /api/capabilities (P3). Presence/usable bools
only ? no
118 * secrets. `id`/`kind` are the backend name (local/gcs/s3/gdrive). */
119 export interface StorageBackendInfo {
120     id: string;
121     kind: string;
122     label: string;
123     usable: boolean;
124     /** Free space in whole GB, or `null` when unknowable (a cloud bucket). */
125     free_gb?: number | null;
126 }
127
128 export interface StorageCapabilities {
129     default_medium?: string;
130     backends?: StorageBackendInfo[];
131 }
132
133 /** POST /api/assets ? 201 (P4). The video was stored into the chosen medium and
registered as a
134     * library asset keyed by the whole-file MD5 (`video_id`). `stored_uri` is the durable

```

```

location to
135     * feed into POST /api/process (a bucket/Drive URI, or a local path for "this
computer"). */
136     export interface AssetResponse {
137         video_id: string;
138         /** The medium backend the asset lives in (local | s3 | gcs | gdrive). */
139         medium: string;
140         /** The durable, MD5-keyed location ? pass as video_path to processVideo(). */
141         stored_uri: string;
142         /** True when a durable copy was made (uploaded ? remote medium); false when
registered in place. */
143         copied: boolean;
144         sport?: string | null;
145         status?: string;
146     }
147
148     /** POST /api/process ? 202 [engine process route]. The job is queued; poll GET
/api/jobs/{job_id}. */
149     export interface ProcessResponse {
150         job_id: string;
151         /** Always "queued" at submit. */
152         status: string;
153         /** Relative poll path, e.g. "/api/jobs/<id>". */
154         poll: string;
155     }
156
157     /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
sequential parts
158     * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
POST /api/process. */
159     export interface UploadInitResponse {
160         upload_id: string;
161         /** Per-part ceiling in MB ? slice the file into parts this big. */
162         max_part_mb: number;
163         next_part: string;
164     }
165     export interface UploadPartResponse {
166         upload_id: string;
167         received_parts: number;
168         received_bytes: number;
169     }
170     export interface UploadCompleteResponse {
171         /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
172         path: string;
173         video_id: string;
174         size_bytes: number;
175     }
176
177     /** GET /api/jobs/{job_id} [engine jobs route]. Two status layers:
178     * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
(success|failed).
179     * A `done` job whose `result.status` === "failed" should surface `result.message`.
180     * A successful job with an explicit empty `play_segments` array is the genuine no-
rallies case. */
181     export interface JobStatus {
182         job_id: string;
183         status: "queued" | "running" | "done" | "failed";
184         /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
percentage. */
185         progress?: string | null;
186         /** The canonical video_id, surfaced top-level on success (also in result.video_id).
*/
187         video_id?: string | null;
188         /** Execution error (worker crash / unexpected) when status === "failed". */
189         error?: string | null;

```

```

190         result?: { status?: string; video_id?: string; message?: string; play_segments?:
RallySegment[] } | null;
191     }
192

```

16. user (2026-07-07T15:08:45.240Z)

```

total 742
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:14 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609    0 Jun 25 23:31 .claude
-rw-r--r-- 1 avidu 197609 94208 Jul  7 20:14 .coverage
-rw-r--r-- 1 avidu 197609   611 Jun 11 10:30 .coveragerc
-rw-r--r-- 1 avidu 197609    69 Jun 10 23:25 .env
-rw-r--r-- 1 avidu 197609 1059 Jul  7 18:29 .gcloudignore
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:35 .git
-rw-r--r-- 1 avidu 197609   947 Jun 25 16:24 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jul  2 08:13 .github
-rw-r--r-- 1 avidu 197609 1998 Jul  2 08:13 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:42 .mypy_cache
drwxr-xr-x 1 avidu 197609    0 May 19 15:58 .pytest_cache
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:32 .ruff_cache
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 .vscode
drwxr-xr-x 1 avidu 197609    0 Jul  7 10:27 __pycache__
drwxr-xr-x 1 avidu 197609    0 Jul  4 00:22 artifacts
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:56 backend
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 backend.db
-rw-r--r-- 1 avidu 197609   6619 Jul  2 11:50 CLAUDE.md
-rw-r--r-- 1 avidu 197609   2735 Jun 16 01:52 config.json
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 deploy
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:17 docs
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 eval_baselines
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 indexer.db
-rw-r--r-- 1 avidu 197609   1999 Jun 25 16:24 mypy.ini
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:14 output
-rw-r--r-- 1 avidu 197609   5413 Jul  4 13:58 pyproject.toml
-rw-r--r-- 1 avidu 197609    240 May 30 14:08 pytest.ini
===SPA web/src/api===
total 56
drwxr-xr-x 1 avidu 197609    0 Jul  5 21:35 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:36 ..
-rw-r--r-- 1 avidu 197609   9211 Jul  2 08:13 client.test.ts
-rw-r--r-- 1 avidu 197609  15801 Jul  5 21:35 client.ts
-rw-r--r-- 1 avidu 197609   2903 Jun 22 23:29 locale.test.ts
-rw-r--r-- 1 avidu 197609   8015 Jul  5 21:35 types.ts
-rw-r--r-- 1 avidu 197609   4401 Jun 21 22:17 upload.test.ts

```

17. assistant (2026-07-07T15:08:49.821Z)

Let me read the backend capabilities handler and the SPA client, plus the config models for `compute\_target`.

18. user (2026-07-07T15:08:52.646Z)

```

1      // Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the
single-origin
2      // appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ?
:8000).
3      // Every call carries a correlation id (header + logs) so a journey is traceable end to
end.
4      import { log, newCorrelationId } from "../lib/log";
5      import type {
6          RallySegment,
7          QueryResponse,

```

```

8      CompileResult,
9      EngineConfig,
10     EngineCapabilities,
11     ProcessResponse,
12     JobStatus,
13     UploadInitResponse,
14     UploadPartResponse,
15     UploadCompleteResponse,
16     VideoSummary,
17     VideosResponse,
18     AnnotationsResponse,
19     AssetResponse,
20 } from "./types";
21
22 /** Resolve the engine API base. Default same-origin `"/api"`. For a SPLIT deploy ? SPA
on
23 * Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel
(HOSTING_PLAN.md §2) ?
24 * set `VITE_API_BASE` at build time to the engine's API base **including the `/api`
path**
25 * (e.g. `https://api.khelsutra.guru/api`), and lock the engine's
`RALLY_CORS_ALLOW_ORIGINS` to the
26 * Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed
so
27 * `` `${base}/process` `` never double-slashes. */
28 export function apiBase(): string {
29     return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
30 }
31
32 const BASE = apiBase();
33
34 export class ApiError extends Error {
35     readonly cid?: string;
36     readonly status?: number;
37     readonly body?: string;
38     constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
39         super(message);
40         this.name = "ApiError";
41         this.cid = opts.cid;
42         this.status = opts.status;
43         this.body = opts.body;
44         if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
45     }
46 }
47
48 interface RequestOpts {
49     method?: string;
50     body?: string;
51     headers?: Record<string, string>;
52     /** Correlation id to thread through this call; auto-generated if absent. */
53     cid?: string;
54 }
55
56 async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
57     const cid = opts.cid ?? newCorrelationId();
58     const url = `${BASE}${path}`;
59     const method = opts.method ?? "GET";
60     log("info", "api.request", { cid, method, url });
61
62     let res: Response;
63     try {
64         res = await fetch(url, {
65             method,
66             body: opts.body,

```

```

67         headers: { "content-type": "application/json", "x-correlation-id": cid,
68         ...(opts.headers ?? {}) },
69     });
70     } catch (err) {
71         // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
72         log("error", "api.network_error", { cid, method, url, error: String(err) });
73         throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
74     }
75     if (!res.ok) {
76         const body = await res.text().catch(() => "");
77         log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
78         throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
79     }
80
81     const data = (await res.json()) as T;
82     log("info", "api.response", { cid, method, url, status: res.status });
83     return data;
84 }
85
86 /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
87 * processing. POST /api/process {video_path} ? 202 {job_id, ...} [engine process
route]. Fast-fails 400
88 * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
89 export async function processVideo(
90     videoPath: string,
91     cid?: string,
92     locale?: string,
93     compute?: string,
94 ): Promise<ProcessResponse> {
95     // `locale` (the UI language) is recorded for PMF analytics (which markets use the
tool). `compute`
96     // is the compute-picker choice ("local" | "cloud") ? where DETECT runs; omitted ? the
server
97     // default. Both optional + omitted when absent (JSON.stringify drops undefined).
98     return request<ProcessResponse>("/process", {
99         method: "POST",
100         body: JSON.stringify({ video_path: videoPath, locale, compute }),
101         cid,
102     });
103 }
104
105 /** Poll a submitted job. GET /api/jobs/{job_id} [engine jobs route]. 404 ? unknown
job_id. */
106 export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
107     return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
108 }
109
110 /** List indexed videos, newest first ? the "my videos" library. GET /api/videos [engine
list_videos route].
111 * Reload-survival: a past match can be reopened without re-processing. Returns [] on
an empty box. */
112 export async function listVideos(cid?: string): Promise<VideoSummary[]> {
113     const data = await request<VideosResponse>("/videos", { method: "GET", cid });
114     return data.videos ?? [];
115 }
116
117 /** The source-video stream URL for annotation playback. GET /api/source/{video_id}
(P8). The engine
118 * serves a frame-identical proxy; labels stay keyed to the master's MD5

```



```

(STUDIO_MEDIA_LIFECYCLE D4). */
119     export function sourceUrl(videoId: string): string {
120         return `${BASE}/source/${encodeURIComponent(videoId)}`;
121     }
122
123     /** Persist finished rally labels as the annotator CSV. POST /api/annotations {video_id,
csv} (P8).
124     * Sent as the byte-compatible CSV (serializeRows) so the engine writes it verbatim
into the corpus. */
125     export async function postAnnotations(videoId: string, csv: string, cid?: string):
Promise<AnnotationsResponse> {
126         return request<AnnotationsResponse>("/annotations", {
127             method: "POST",
128             body: JSON.stringify({ video_id: videoId, csv }),
129             cid,
130         });
131     }
132
133     /** Store a video into the user's chosen medium + register it as a library asset (P4,
D14).
134     * POST /api/assets {uploaded_path? | source_uri?, medium_uri, sport?} ? 201 {video_id,
stored_uri}.
135     * Exactly ONE source: an `uploadedPath` (a just-uploaded local file ? a durable
MD5-keyed copy is
136     * put into the medium, the local upload kept as the working copy) OR a `sourceUri` (a
video already
137     * at rest, registered as-is). Feed `stored_uri` into processVideo(). Fast-fails 400
(bad medium /
138     * both-or-neither source), 403 (hosted posture), 404 (missing file), 507 (no space),
502 (put) ?
139     * surfaced as ApiError. */
140     export async function createAsset(
141         args: { uploadedPath?: string; sourceUri?: string; mediumUri: string; sport?: string
},
142         cid?: string,
143     ): Promise<AssetResponse> {
144         return request<AssetResponse>("/assets", {
145             method: "POST",
146             body: JSON.stringify({
147                 uploaded_path: args.uploadedPath,
148                 source_uri: args.sourceUri,
149                 medium_uri: args.mediumUri,
150                 sport: args.sport,
151             }),
152             cid,
153         });
154     }
155
156     /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[engine query route]. */
157     export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
158         const data = await request<QueryResponse>("/query", {
159             method: "POST",
160             body: JSON.stringify({ video_id: videoId }),
161             cid,
162         });
163         return data.play_segments ?? [];
164     }
165
166     // Compile-job poll cadence (mirrors useIngestJob): a long ffmpeg stitch must not block
the request.
167     const COMPILE_POLL_MIN_MS = 1000;
168     const COMPILE_POLL_MAX_MS = 5000;
169     const COMPILE_POLL_BACKOFF = 1.5;

```

```

170    // A wedged 'running' compile won't poll forever ? but a long reel can legitimately
171    // the control plane stitches in-process behind a single drain lane, so a compile can
172    // a running detection before it even starts (same 2026-07-05 rationale as the ingest
173    // ceiling).
174    const COMPILE_POLL_CEILING_MS = 60 * 60 * 1000;
175    /** Stitch selected rallies into one reel. ASYNC (#329): POST /api/compile returns 202 +
176    * the ffmpeg stitch runs on the worker and we POLL GET /api/jobs/{id} for the reel ? a
177    * would otherwise exceed the ~100 s edge/proxy timeout (524) while the reel actually
178    * with the job's CompileResult on success; throws ApiError on a failed/timed-out
179    * `locale` (the UI language) localizes the reel watermark engine-side (omitted when
180    * absent). */
181    export async function compileReel(
182        videoId: string,
183        segmentIds: number[],
184        outputFilename: string,
185        cid?: string,
186        locale?: string,
187    ): Promise<CompileResult> {
188        const c = cid ?? newCorrelationId();
189        const submitted = await request<ProcessResponse>("/compile", {
190            method: "POST",
191            body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
192            outputFilename, locale }),
193            cid: c,
194        });
195        return pollCompileJob(submitted.job_id, c);
196    }
197    /** Poll a submitted compile job to its terminal state. Returns the reel result on
198    * success; throws an
199    * ApiError for an engine job-failure, a success-shaped FAILURE (done but the stitch
200    * failed / no URL),
201    * or a poll timeout ? so the caller's existing classifyError/errorText mapping
202    * surfaces it like ingest. */
203    async function pollCompileJob(jobId: string, cid: string): Promise<CompileResult> {
204        const startedAt = Date.now();
205        let delay = COMPILE_POLL_MIN_MS;
206        for (;;) {
207            const job = await getJob(jobId, cid);
208            if (job.status === "done") {
209                const result = (job.result ?? {}) as CompileResult;
210                if (result.status === "failed" || !result.download_url) {
211                    // A real stitch failure carries an engine message ? surface it as an `engine`
212                    detail (status +
213                    // body set) so classifyError shows it verbatim, NOT the no-status "engine
214                    unreachable" fallback.
215                    const msg = result.message || "Highlight stitching failed.";
216                    throw new ApiError(msg, { cid, status: 502, body: msg });
217                }
218                return result;
219            }
220            if (job.status === "failed") {
221                const msg = job.error || "Compile job failed.";
222                throw new ApiError(msg, { cid, status: 502, body: msg });
223            }
224            if (Date.now() - startedAt > COMPILE_POLL_CEILING_MS) {
225                throw new ApiError("Compile timed out.", { cid, status: 504, body: "Compile timed
226                out." });
227            }
228            delay = Math.min(delay * 2, COMPILE_POLL_MAX_MS);
229            await sleep(delay);
230        }
231    }

```

```

220     }
221     await new Promise((resolve) => setTimeout(resolve, delay));
222     delay = Math.min(delay * COMPILE_POLL_BACKOFF, COMPILE_POLL_MAX_MS);
223   }
224 }
225
226 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [engine highlights route]. */
227 export function highlightUrl(videoId: string, filename: string): string {
228   return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
229 }
230
231 /** Read engine config (for the stitching floor). GET /api/config [engine config route].
*/
232 export async function getConfig(cid?: string): Promise<EngineConfig> {
233   return request<EngineConfig>("/config", { method: "GET", cid });
234 }
235
236 /** What THIS box can do ? drives the first-run wizard. GET /api/capabilities
(PACKAGING_PLAN P2c). */
237 export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
238   return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
239 }
240
241 /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[engine compile floor]. */
242 export function minShotCount(cfg: EngineConfig): number | undefined {
243   return cfg.stitching?.min_shot_count;
244 }
245
246 // ?? chunked upload (backend/api/uploads.py) ??????????????????????????????
247 // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
248 // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
249
250 /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[backend/api/uploads.py]. */
251 export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
252   return request<UploadInitResponse>("/upload/init", {
253     method: "POST",
254     body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
255     cid,
256   });
257 }
258
259 /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[backend/api/uploads.py]. */
260 export async function uploadPart(
261   uploadId: string,
262   index: number,
263   chunk: Blob,
264   cid?: string,
265   signal?: AbortSignal,
266 ): Promise<UploadPartResponse> {
267   const c = cid ?? newCorrelationId();
268   const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
269   log("info", "api.request", { cid: c, method: "PUT", url });
270   let res: Response;
271   try {
272     res = await fetch(url, {
273       method: "PUT",
274       body: chunk,

```

```

275         headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
276         signal,
277     });
278 } catch (err) {
279     log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
280 });
281     throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
282 }
283 if (!res.ok) {
284     const body = await res.text().catch(() => "");
285     log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
body: body.slice(0, 500) });
286     throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
body });
287 }
288 log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
289 return (await res.json()) as UploadPartResponse;
290 }
291 /** Finalize: returns {path, video_id} ? feed `path` to processVideo()
[backend/api/uploads.py]. */
292 export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
293     return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
294         method: "POST",
295         body: JSON.stringify({ total_parts: totalParts }),
296         cid,
297     });
298 }
299
300 /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[backend/api/uploads.py]. */
301 export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
302     await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });
303 }
304
305 export interface UploadProgress {
306     sentBytes: number;
307     totalBytes: number;
308 }
309
310 /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
311 * On ANY failure or abort it best-effort DELETES the server-side partial, then
rethrows so the
312 * caller can classify it (the engine's 400/413 detail survives). Returns {path,
video_id}.
313 * Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
recovery is a
314 * fresh init (re-pick the file), which the UI's retry does. */
315 export async function uploadFile(
316     file: File,
317     opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
= {},
318 ): Promise<UploadCompleteResponse> {
319     const cid = opts.cid ?? newCorrelationId();
320     const init = await uploadInit(file.name, file.size, cid);
321     // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
to NaN (which
322     // would silently upload zero parts) ? fall back to the engine's 95 MB default.
323     const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
init.max_part_mb : 95;
324     const partBytes = partMb * 1024 * 1024;

```

```

325     const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
326     log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
327     let sent = 0;
328     try {
329         for (let i = 0; i < totalParts; i++) {
330             if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
331             const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
332             await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
333             sent += chunk.size;
334             opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
335         }
336         const done = await uploadComplete(init.upload_id, totalParts, cid);
337         log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
338         return done;
339     } catch (err) {
340         log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
341         await uploadAbort(init.upload_id, cid).catch(() => {});
342         throw err;
343     }
344 }
345

```

19. user (2026-07-07T15:08:53.335Z)

```

235:     _cloud_available,
267:     # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268:     # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
364:@app.get("/api/capabilities")
365:def capabilities(request: Request):
371:     deployment = getattr(cfg, "deployment", None)
397:     "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
399:     "deployment": {
400:         "profile": getattr(deployment, "profile", "") or "",
401:         "compute_target": getattr(deployment, "compute_target", "") or "",
404:         "cloud_available": _cloud_available(cfg),
766:         "storing into a custom medium is disabled on this hosted deployment "
1237:     ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?

```

20. assistant (2026-07-07T15:08:56.488Z)

Let me read the backend capabilities handler and the deployment config model.

21. user (2026-07-07T15:08:58.492Z)

```

364     @app.get("/api/capabilities")
365     def capabilities(request: Request):
366         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
367         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
368         value. Best-effort + never raises (the wizard degrades gracefully)."""
369         cfg = request.app.state.config
370         indexing = getattr(cfg, "indexing", None)
371         deployment = getattr(cfg, "deployment", None)
372         ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
373         ffprobe = shutil.which(ffprobe_bin())
374         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
375         wasb = getattr(indexing, "wasb", None)

```

```

376     weights_path = (
377         os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
378     )
379     # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
380     # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
381     # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
382     # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
383     nvidia_smi = shutil.which("nvidia-smi")
384     return {
385         "api_version": API_VERSION,
386         "engine_version": _engine_version(),
387         "segmenters": sorted(segmenter_registry.list_available().keys()),
388         "default_segmenter": getattr(indexing, "default_segmenter", None),
389         "ffmpeg": {
390             "ffmpeg": bool(ffmpeg),
391             "ffprobe": bool(ffprobe),
392             "ffmpeg_path": ffmpeg,
393             "ffprobe_path": ffprobe,
394         },
395         "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
396         "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
397         "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
398         **uploads_api.upload_caps(cfg), # #480 item 5: the SPA pre-flight reads
max_upload_mb
399         "deployment": {
400             "profile": getattr(deployment, "profile", "") or "",
401             "compute_target": getattr(deployment, "compute_target", "") or "",
402             # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
403             # the "run in the cloud" toggle only when true (else it's local-only).
404             "cloud_available": _cloud_available(cfg),
405         },
406         # P3: which storage media THIS box can store INTO (local always; s3/gcs if
SDK+config;
407         # gdrive if a mount resolves). The P5 medium picker renders only usable
backends. No secrets.
408         "storage": storage_capabilities(
409             getattr(cfg, "storage", None), local_free_path=output_base(cfg)
410         ),
411     }
412
413
414     def _friendly_video_name(filepath: str) -> str:
415         """A human name for a video from its stored path/URI ? the filename stem
(``~/match_2026.mp4`` ?
416         ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
(rsplit on ``/``).
417         Falls back to the whole ``filepath`` if there's no basename."""
418         base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
419         stem = os.path.splitext(base)[0]
420         return stem or base or str(filepath or "")
421
422
423     def _video_dims(validation_results: Any) -> "tuple[Optional[float], Optional[str]]":

```

22. user (2026-07-07T15:09:00.766Z)

```

backend/config/loader.py:200:         ct = merged["deployment"].get("compute_target")
backend/config/loader.py:203:         # An active profile with no explicit compute_target
(e.g. config.json blanked it
backend/config/loader.py:205:         # active profile's compute_target (it stays

```

```

meaningful for the M2+ ComputeBackend).
backend/config/loader.py:206: merged["deployment"]["compute_target"] = canonical
backend/config/loader.py:209: "[CONFIG] deployment profile %r normally uses
compute_target=%r but "
backend/config/loader.py:216: "[CONFIG] deployment profile ACTIVE: %s
(compute_target=%s)",
backend/config/loader.py:218: merged["deployment"].get("compute_target"),
backend/config/loader.py:254:# local storage and the compute_target, so this stays the handful
of knobs a fresh, non-technical
backend/config/models.py:471: compute_target: Literal["", "local_gpu", "cloud_run",
"cpu_mock"] = ""
backend/config/models.py:473: # bucket-poll CONCURRENTLY when compute_target=="cloud_run".
Each is an I/O-bound Cloud Run Job
backend/config/presets.py:30:# Each profile maps 1:1 to its canonical compute_target. Used to
apply the preset and to warn
backend/config/presets.py:31:# when config.json explicitly overrides compute_target into a state
inconsistent with the profile.
backend/config/presets.py:39:# that EXIST today ? M1 is pure/additive: the new
`deployment.{profile,compute_target}` plus
backend/config/presets.py:45: "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend/config/presets.py:52: "deployment": {"profile": "cloud-serving",
"compute_target": "cloud_run"},
backend/config/presets.py:75: "deployment": {"profile": "cpu-mock", "compute_target":
"cpu_mock"},
Binary file backend/config/__pycache__/loader.cpython-313.pyc matches
Binary file backend/config/__pycache__/models.cpython-313.pyc matches
Binary file backend/config/__pycache__/presets.cpython-313.pyc matches
===DEPLOYMENT CONFIG MODEL===
49: conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
78: conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
454:class DeploymentConfig(BaseModel):
455: """Deployment profile ? one switch that selects a coherent bundle of compute + storage
458: **Pure / additive:** the empty default profile (``"``) applies NO preset, so the
pipeline
459: behaves exactly as before this section existed. A profile is opt-in (``config.json``
460: ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
462: these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
467: # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
468: profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
469: # Where the core runs. Set by the profile preset; overridable in config.json. Inert in
M1
471: compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
473: # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound Cloud
Run Job

```

23. assistant (2026-07-07T15:10:33.478Z)

Let me verify the hosted default segmenter, whether the SPA ever sends a segmenter, and the useIngestJob submit path.

24. user (2026-07-07T15:10:38.085Z)

```

===cloud-serving preset + default_segmenter===
4:config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
27:PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28:COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
34: "cloud-serving": "cloud_run",
51: "cloud-serving": {
52:     "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
63:     # out + canaried (DEPLOY_REPORT_cloud-serving_gpu-resident-nvdec_2026-07-07).
This makes
133: - ``K_SERVICE`` set (Cloud Run) ? ``"cloud-serving"``
139: imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe

```

```

passes
144:         return "cloud-serving"
=== config.json default_segmenter ===
26:         "default_segmenter": "gemini",
47:         "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
59:         "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
=== deploy configs referencing default_segmenter/profile/compute_target ===
deploy/cloudrun/gen_service_yaml.py:17:server, so one image serves every config shape
(config.json wins over the cloud-serving preset ?
deploy/cloudrun/gen_service_yaml.py:36: * `indexing.default_segmenter=wasb_hybrid` ? the engine
default is yolo_hybrid, so an unpinned
deploy/cloudrun/gen_service_yaml.py:66:         "deployment": {"profile": "cloud-serving",
"compute_target": "cloud_run"},
deploy/cloudrun/gen_service_yaml.py:71:         "default_segmenter": args.default_segmenter,
deploy/cloudrun/gen_service_yaml.py:130:         - name: DEPLOYMENT_PROFILE
deploy/cloudrun/gen_service_yaml.py:131:         value: cloud-serving
deploy/cloudrun/post_deploy_smoke.ps1:36: --set=deployment.compute_target=local_gpu",
deploy/cloudrun/README.md:3:The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
deploy/cloudrun/README.md:56: on-GPU. **The cloud-serving preset default** (validated: offline
golden gate @ SERVED F1 0.574 ?
deploy/cloudrun/README.md:60:**How the knob reaches the worker.** The worker Job runs with NO
`DEPLOYMENT_PROFILE`, so it does
deploy/cloudrun/README.md:61:**not** apply the cloud-serving preset itself. The control plane
(which *does* have the profile)
deploy/cloudrun/README.md:88:The control plane runs with the **cloud-serving** profile
(`DEPLOYMENT_PROFILE=cloud-serving` ?
deploy/cloudrun/README.md:89:`compute_target=cloud_run`). `get_compute_backend(config)` then
builds `CloudRunCompute` from env:
deploy/cloudrun/README.md:91:export DEPLOYMENT_PROFILE=cloud-serving
deploy/cloudrun/README.md:94:# running in-process; the container runs the SAME worker INLINE
(compute_target forced local_gpu).
deploy/cloudrun/README.md:100:0-rally "success" **after** the paid GPU pass; the cloud-serving
startup checks now fail this at
deploy/cloudrun/README.md:111:cloud-serving config.json into a startup wrapper, sets every flag
below, pins `maxScale=1`
deploy/cloudrun/README.md:126: --set-env-vars DEPLOYMENT_PROFILE=cloud-
serving,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
deploy/cloudrun/README.md:139:Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
deploy/cloudrun/README.md:145:- The dispatched container **must run inline** ? `CloudRunCompute`
forces `compute_target=local_gpu`
deploy/cloudrun/README.md:165: dispatcher's two forced `--set`s:
`deployment.compute_target=local_gpu`,
deploy/gcp/README.md:205: "indexing": { "default_segmenter": "wasb_hybrid", "detector_impl":
"native" },

```

25. user (2026-07-07T15:10:40.150Z)

```

===any 'segmenter' in web/src===
src/api/types.ts:20: /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
src/api/types.ts:22: /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
src/api/types.ts:95: segmenters?: string[];
src/api/types.ts:96: default_segmenter?: string;
src/components/SetupWizard.test.tsx:24: default_segmenter: "motion",
src/components/SetupWizard.test.tsx:30: default_segmenter: "gemini",
src/components/SetupWizard.test.tsx:38: default_segmenter: "wasb_hybrid",
src/components/SetupWizard.test.tsx:54: const detector = caps?.default_segmenter ?? "motion";
===useIngestJob submit signature/impl===
1:// Drives the front-half ingest: submit a video to POST /api/process, then poll GET
/api/jobs/{id}
10:import { getJob, processVideo } from "../api/client";
31: compute?: string;
37: | { status: "submitting"; displayName?: string }
69: compute: parsed.compute ? String(parsed.compute) : undefined,

```



```

125: // Keep the latest onLoaded without making submit depend on it (a parent re-render won't
restart polls).
237: const submit = useCallback(
238:   (rawUri: string, locale?: string, compute?: string, displayName?: string) => {
251:     setPhase({ status: "submitting", ...(displayName ? { displayName } : {}) });
252:     log("info", "ingest.submit_start", { cid, uri, display_name: displayName });
265:     const resp = await processVideo(uri, cid, locale, compute);
268:     failSubmit(classifyError(err), "ingest.submit_error");
272:     const active: ActiveIngestJob = { jobId, uri, cid, startedAt, locale, compute,
displayName };
315: const busy = phase.status === "submitting" || phase.status === "polling";
316: return { phase, submit, cancel, reset, busy };

```

26. assistant (2026-07-07T15:11:47.596Z)

The SPA never sends a segmenter (confirmed) ? the backend always uses its configured `default\_segmenter`, which for the hosted build is `wasb\_hybrid` (weights-backed). Let me verify the profile auto-detection (does a real Cloud Run box always get `compute\_target=cloud\_run`?) and `get\_compute\_backend`'s default-off behavior, to size the case (c) asymmetry.

27. user (2026-07-07T15:11:49.609Z)

```

100
101 def _cuda_available() -> bool:
102     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
103     ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
104     try:
105         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
106         # and optional). Only reached when a caller opts into probe_cuda.
107         import torch
108
109         return bool(torch.cuda.is_available())
110     except Exception:
111         return False
112
113
114 def probe_cuda_available() -> bool:
115     """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
116
117     Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
118     that *wants* the GPU heads-up can pass the result in without the startup module
importing
119     torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
120     (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
121     treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
122     error that could false-fail."""
123     return _cuda_available()
124
125
126 def suggest_profile(
127     env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
128 ) -> str:
129     """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
130
131     Signals, in priority order:
132
133     - ``K_SERVICE`` set (Cloud Run) ? ``"cloud-serving"``
134     - ``CI`` truthful ? ``"cpu-mock"``

```

```

135         - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
136         - otherwise                               ? ``""`` (no strong suggestion)
137
138     ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
139     imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
140     ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
141     """
142     e = os.environ if env is None else env
143     if e.get("K_SERVICE"):
144         return "cloud-serving"
145     if _truthy(e.get("CI")):
146         return "cpu-mock"
147     if probe_cuda and _cuda_available():
148         return "truly-local"
149     return ""
150

```

28. user (2026-07-07T15:11:53.692Z)

```

===get_compute_backend===
backend/compute/__init__.py:42:def get_compute_backend(
42:def get_compute_backend(
---
backend/compute/__init__.py:42:def get_compute_backend(
===registered segmenters (names) + yolo_hybrid?===
backend/pipeline/segmenters/__init__.py:7:from backend.pipeline.segmenters.yolo_hybrid import
HybridYoloSegmenter
backend/pipeline/segmenters/base.py:66:     def register(self, segmenter_cls:
Type[BasePlaySegmenter]):
backend/pipeline/segmenters/base.py:81:         """Returns a dict of registered segmenter names
and their descriptions."""
backend/pipeline/segmenters/fusion_hybrid.py:15: via the pre-existing yolo_hybrid ?
person_detector chain; the lazy part is the model.)
backend/pipeline/segmenters/fusion_hybrid.py:24:and hands off identically to `wasb_hybrid`.
Registered as ``fusion_hybrid``, NOT the default
backend/pipeline/segmenters/fusion_hybrid.py:243:segmenter_registry.register(FusionSegmenter)
backend/pipeline/segmenters/gemini.py:707:segmenter_registry.register(GeminiPlaySegmenter)
backend/pipeline/segmenters/motion.py:214:         value. Matches yolo_hybrid/gemini, which pass
``server=None, receiver=None``.
backend/pipeline/segmenters/motion.py:243:segmenter_registry.register(MotionPlaySegmenter)
backend/pipeline/segmenters/tracknet_hybrid.py:48:segmenter_registry.register(TrackNetHybridSegm
enter)
backend/pipeline/segmenters/trajectory_hybrid.py:18:This base is deliberately NOT registered;
only concrete subclasses register.
backend/pipeline/segmenters/trajectory_hybrid.py:296:         # their own frame buffering), so
unlike yolo_hybrid we don't pre-chunk.
backend/pipeline/segmenters/wasb_hybrid.py:68:segmenter_registry.register(WasbHybridSegmenter)
backend/pipeline/segmenters/yolo_hybrid.py:17:The registry key stays "yolo_hybrid" for back-
compat with existing configs and the
backend/pipeline/segmenters/yolo_hybrid.py:57:         return "yolo_hybrid"
backend/pipeline/segmenters/yolo_hybrid.py:304:         detector="yolo_hybrid",
backend/pipeline/segmenters/yolo_hybrid.py:425:         f"yolo_hybrid AI handoff produced "
no verdict: all {handoff_failed} candidate "
backend/pipeline/segmenters/yolo_hybrid.py:515:         message=f"yolo_hybrid:
invalid/unreadable video duration ({video_duration})."
backend/pipeline/segmenters/yolo_hybrid.py:596:segmenter_registry.register(HybridYoloSegmenter)

```

29. assistant (2026-07-07T15:14:00.519Z)

`suggest\_profile` only *suggests* (never auto-applies), and the Cloud Run deploy explicitly sets
`DEPLOYMENT\_PROFILE=cloud-serving` and pins `default\_segmenter=wasb\_hybrid`. Now let me verify
the weights-probe parity with the actual runners (a false-negative would wrongly force paid

cloud) ? especially TrackNet, where the probe checks only config with no env override.

30. user (2026-07-07T15:14:04.393Z)

```
===models.py wasb/tracknet weights_path defaults + substrate===
46:class TrackNetConfig(BaseModel):
51:    weights_path: str = ""
69:    """Typed config for the live WASB shuttle substrate (indexing.wasb).
80:    weights_path: str = (
168:class FusionConfig(BaseModel):
179:    # Which trajectory substrate seeds the windows (shuttle stays primary).
180:    substrate: Literal["wasb", "tracknet"] = "wasb"
182:    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
202:class IndexingConfig(BaseModel):
203:    default_segmenter: str = "yolo_hybrid"
297:    @field_validator("default_segmenter")
```

31. user (2026-07-07T15:14:06.660Z)

```
===WASB runner weight resolution===
backend/pipeline/detectors/native_wasb_runner.py:17:    (`WASB_WEIGHTS` / `WASB_REPO_DIR` /
`WASB_PYTHON` / `WASB_SCRATCH`), then
backend/pipeline/detectors/native_wasb_runner.py:87:    weights_path: str = "" # REQUIRED (env
WASB_WEIGHTS or indexing.wasb.weights_path)
backend/pipeline/detectors/native_wasb_runner.py:113:        env = os.environ.get
backend/pipeline/detectors/native_wasb_runner.py:120:
weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
backend/pipeline/detectors/native_wasb_runner.py:202:        if not self.cfg.weights_path:
backend/pipeline/detectors/native_wasb_runner.py:204:            "WASB weights path is empty
? set WASB_WEIGHTS (or indexing.wasb.weights_path)."
backend/pipeline/detectors/native_wasb_runner.py:206:        weights =
os.path.expanduser(self.cfg.weights_path)
backend/pipeline/detectors/native_wasb_runner.py:298:        weights =
os.path.expanduser(self.cfg.weights_path)
backend/pipeline/detectors/wasb_runner.py:45:    weights_path: str = (
backend/pipeline/detectors/wasb_runner.py:46:        "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
backend/pipeline/detectors/wasb_runner.py:71:        weights_path=w.weights_path,
backend/pipeline/detectors/wasb_runner.py:183:        f"--weights
{wsl_tilde_quote(self.cfg.weights_path)} "
backend/pipeline/detectors/tracknet_runner.py:43:    weights_path: str = "" # WSL path to the
.h5/.keras weights (REQUIRED)
backend/pipeline/detectors/tracknet_runner.py:61:        weights_path=tn.weights_path,
backend/pipeline/detectors/tracknet_runner.py:142:    if not self.cfg.weights_path:
backend/pipeline/detectors/tracknet_runner.py:144:        "TrackNet weights_path is not
set. Set indexing.tracknet.weights_path "
backend/pipeline/detectors/tracknet_runner.py:197:        f"--model_weights
{wsl_tilde_quote(self.cfg.weights_path)} "
```

32. assistant (2026-07-07T15:14:47.838Z)

Let me confirm the `WasbConfig.weights\_path` default and whether `yolo\_hybrid` (the engine-default segmenter, absent from the guardrail set) needs pre-baked weights.

33. user (2026-07-07T15:14:49.329Z)

```
46    class TrackNetConfig(BaseModel):
47        wsl_distro: str = "Ubuntu"
48        repo_dir: str = "~/models/TrackNetV4"
49        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50        conda_env: str = "TrackNetV4"
51        weights_path: str = ""
52        queue_length: int = 5
53        stage_in_wsl: bool = True
```

```

54     wsl_stage_dir: str = "~/clips"
55     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56     velocity_threshold: float = 0.02
57     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58     # decoded frame cache) are deleted automatically ? they are pure intermediates,
59     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61     keep_frames: bool = False
62     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
63     # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
64     # 0 = no extra floor (the headroom guard still applies).
65     min_free_gb: float = 0.0
66
67
68     class WasbIndexConfig(BaseModel):
69         """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72         these knobs actually take effect ? previously this whole block was silently dropped
73         because nested config sections defaulted to extra='ignore'.
74         """
75
76         wsl_distro: str = "Ubuntu"
77         repo_dir: str = "~/models/WASB-SBDT"
78         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79         conda_env: str = "wasb"
80         weights_path: str = (
81             "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82         )
83         sport: str = "badminton"
84         wsl_stage_dir: str = "~/clips"
85         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86         velocity_threshold: float = 0.02
87         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
88         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91         device: str = "cuda" # auto | cuda | mps | cpu
92         python_bin: str = (
93             "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
94         )
95         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache

```

34. user (2026-07-07T15:14:53.715Z)

===does yolo_hybrid need baked weights? (person detector weight handling)===

9:Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted

===person_detector weight source===

backend/pipeline/detectors/person_detector.py

backend/pipeline/detectors/__pycache__/person_detector.cpython-313.pyc

backend/pipeline/segmenters/yolo_hybrid.py

backend/pipeline/segmenters/__pycache__/yolo_hybrid.cpython-313.pyc

===what does gen_service_yaml pin + is default_segmenter always set for hosted?===

* No secrets here: bucket/project/aud/team-domain are the same non-secret identifiers the committed runlogs carry; the Gemini key stays in Secret Manager (the alpha runs `skip\_ai\_handoff=true` and never needs it).

Baked config pins (CONTROL_PLANE_DURABLE_REBUILD P1/D5 ? the live rev-00011 posture, learned on the 2026-07-05 owner CUJ; before this, regenerating the manifest silently DROPPED both):

* `indexing.default\_segmenter=wasb\_hybrid` ? the engine default is yolo_hybrid, so an unpinned

```

cloud job ran the wrong segmenter (#479).
* `security.max_part_mb=24` ? upload parts must fit under the ~32 MiB HTTP/1 body cap at the
  Google Front End (#480); the engine default (95) gets every part 413'd before FastAPI runs.
  NOT fixed via `use-http2`: h2c requires the container itself to speak cleartext HTTP/2,
  which uvicorn does not (spec D6) ? the annotation would break the service.
* `--memory` defaults to 8Gi (spec D2) ? compile still pulls the gs:// source into the
  RAM-backed /tmp to stitch, so the instance must hold the largest allowed upload with
  headroom; >8Gi would force 4 vCPU on Cloud Run for no current need.
"""

```

```

from __future__ import annotations

```

```

import argparse
import base64
import json
import sys

```

```

DEFAULT_PROJECT = "gen-lang-client-0306026826"
DEFAULT_REGION = "asia-southeast1"
DEFAULT_SERVICE = "rally-control-plane"
DEFAULT_SERVING_BUCKET = "gs://khelsutra-rally-serving"
DEFAULT_CF_TEAM_DOMAIN = "https://khelsutra.cloudflareaccess.com"
DEFAULT_CF_ACCESS_AUD = "c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b"

```

```

def build_config(args: argparse.Namespace) -> dict:
    """The app config the wrapper materializes at RALLY_APP_HOME/config.json."""
    bucket = args.serving_bucket.rstrip("/")
    return {
        "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
        "indexing": {
            # WASB-only alpha: candidate windows ARE the rallies; no Gemini key in this service.
            "skip_ai_handoff": True,
            # rev-00011 pin (#479): the engine default is yolo_hybrid ? pin the validated one.
            "default_segmenter": args.default_segmenter,
        },
        "storage": {
            "backend": "gcs",
            "output_root": bucket,
        }
    }

```

35. assistant (2026-07-07T15:16:33.548Z)

The WASB probe matches the native runner's resolution (env `WASB\_WEIGHTS` ?
`indexing.wasb.weights\_path`, default `~/models/WASB-SBDT/...`), and the hosted build pins
`default\_segmenter=wasb\_hybrid`, so the guardrail engages on the real hosted CP. One last
critical check: does `get\_compute\_backend(cfg, target\_override="cloud\_run")` actually force a
Cloud Run dispatch even when the box's own `compute\_target` is `local\_gpu`/`""` (case b'/c)? If
it ignores the override, the SPA sending `cloud` would silently fall through to in-process.

36. user (2026-07-07T15:16:35.606Z)

```

42     def get_compute_backend(
43         config: Any,
44         *,
45         run_client: Optional[Any] = None,
46         policy: ExecutionPolicy = DEFAULT_POLICY,
47         token: Optional[str] = None,
48         target_override: Optional[str] = None,
49     ) -> Optional[ComputeBackend]:
50         """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
51         in-process (the default-OFF path).
52
53         Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its

```

```

54         Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
55         ``CLOUD_RUN_REGION``
56         / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
56         pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
57
58         ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
59         per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
60         server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
61         """
62         target = (
63             target_override
64             if target_override is not None
65             else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
66         )
67         if target != "cloud_run":
68             return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
69
70         # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
71         from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
72
73         # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
74         # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
so a
75         # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild $4.3; the
old
76         # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
an
77         # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
78         if policy is DEFAULT_POLICY:
79             max_wait = getattr(
80                 getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
81             )
82             if max_wait:
83                 policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
84
85         job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
86         region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
87         project = os.environ.get("GOOGLE_CLOUD_PROJECT")
88         client = run_client if run_client is not None else GoogleCloudRunJobClient()
89         return CloudRunCompute(
90             run_client=client,
91             job_name=job_name,
92             region=region,
93             project=project,
94             storage_config=_storage_dict(config),
95             policy=policy,
96             token=token,
97         )
98

```

37. assistant (2026-07-07T15:17:29.722Z)

I've fully traced the cross-repo contract end-to-end against the actual code on both sides. Let me record the final verification of the key invariant before concluding.

The decisive facts I verified:

1. ****Same gate on both sides.**** The SPA renders the cloud toggle iff ``capabilities.deployment.cloud_available == true``, and that value is literally

``_cloud_available(cfg)`` (backend ``main.py:404``). The backend's new fail-fast fires only when ``_cloud_available(cfg)`` is true (``orchestration_new.py:490``). So whenever the backend returns "Choose 'Run in cloud'", the SPA is guaranteed to be showing a cloud toggle ? the message is always actionable, never a dead-end.

2. `**`get_compute_backend(cfg, target_override="cloud_run")`` honors the override regardless of the box's configured ``compute_target`` (``backend/compute/__init__.py:62-68``). So in case (b?)/(c), when the user picks cloud, a real Cloud Run dispatch happens ? there is no silent "SPA sends cloud ? backend runs in-process" fallback.

3. `**SPA never sends a `segmenter``; the backend always uses its configured ``default_segmenter``, which the hosted deploy pins to ``wasb_hybrid`` (weights-backed), so the guardrail engages on the real hosted CP. The weights probe matches the native WASB runner's resolution (``WASB_WEIGHTS`` env ? ``indexing.wasb.weights_path``) and the TrackNet runner's (config-only, no env), so no false-negative that would wrongly force paid cloud.

4. `**Every matrix cell is improved, none regressed.**` (a) hosted ? SPA defaults cloud, dispatches; manual-local ? fail-fast ? cloud offered. (b)/(b?) local appliance ? picker hidden or local default, weights-present escape hatch runs free in-process. (c) env-wired: W=true ? free in-process (optimal); W=false ? clear recoverable fail-fast pointing at the cloud toggle the SPA is showing (strictly better than the baseline cryptic deep failure).

The one asymmetry the task flags ? SPA defaults-to-cloud on the narrow ``compute_target=="cloud_run"`` while the backend guards on the broader ``_cloud_available`` ? does not create a dead-end or contradiction. In case (c) it only means the first submit on a weightless env-wired box fails clearly and recoverably instead of auto-selecting paid cloud; and defaulting such a box to local is the correct choice for the (more common) weights-present sub-case, since the SPA cannot distinguish W=true from W=false via ``compute_target`` alone. The mildly generic "hosted server / control-plane" wording in that rare case is a nit, not a behavioral defect. The only true dead-end reachable (cloud_available via env but no ``storage.input_root``, plus a local upload) is pre-existing, orthogonal to this weights guardrail, and configured-away on the real hosted CP.

No real, concrete cross-repo-contract defect exists in this change.

38. user (2026-07-07T15:17:29.982Z)

Structured output provided successfully